

Managing Environment Variables and Flavors in Flutter: A Guide for Android & iOS

7 min read · Sep 14, 2024



Sunday David

Follow



Listen



Share

Establishing multiple environments in app development is essential for thorough testing, smooth deployment, and the overall strength of the final product. By separating development, staging, and production environments, developers can perform in-depth testing, identify and fix bugs, and adjust configurations tailored to each setting. This approach supports concurrent development, client presentations, and user acceptance testing, ensuring the application meets expectations and functions efficiently. Moreover, it facilitates quick rollbacks, adherence to industry regulations, and scalability assessments. Ultimately, multiple environments lead to a more dependable and robust product, improving the entire development process.

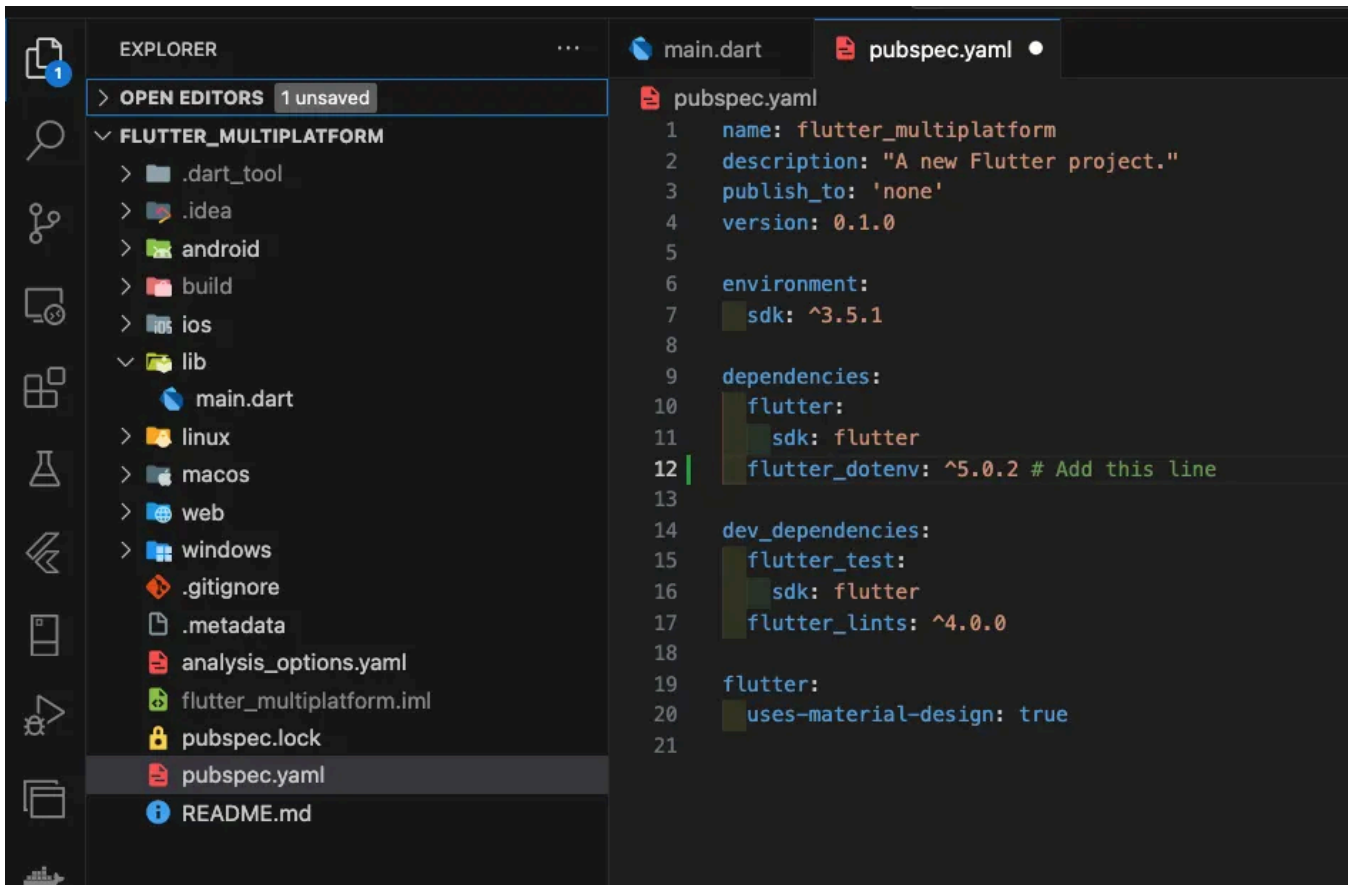
We'll use the [flutter_dotenv](#) flutter package to expose environment variables to your Flutter app, which supports iOS, Android, and the web.

Let's get started by creating the Flutter app. I'll assume you already have [Flutter](#) and [Dart](#) installed on your machine. Follow [these steps to create an empty flutter project \(say flutter_multiplatform\)](#) using VS Code.

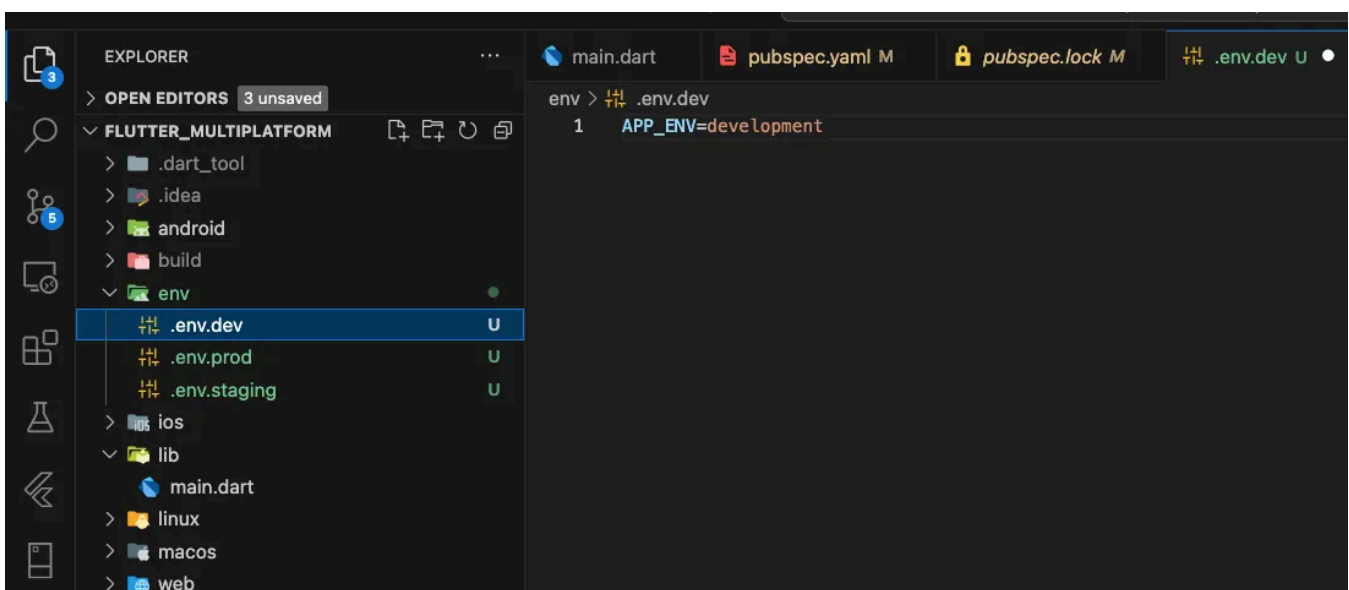
Yay! Your app should be ready to run using the command below in your terminal, and select either simulator or emulator (assuming you're using a MacBook):

```
flutter run
```

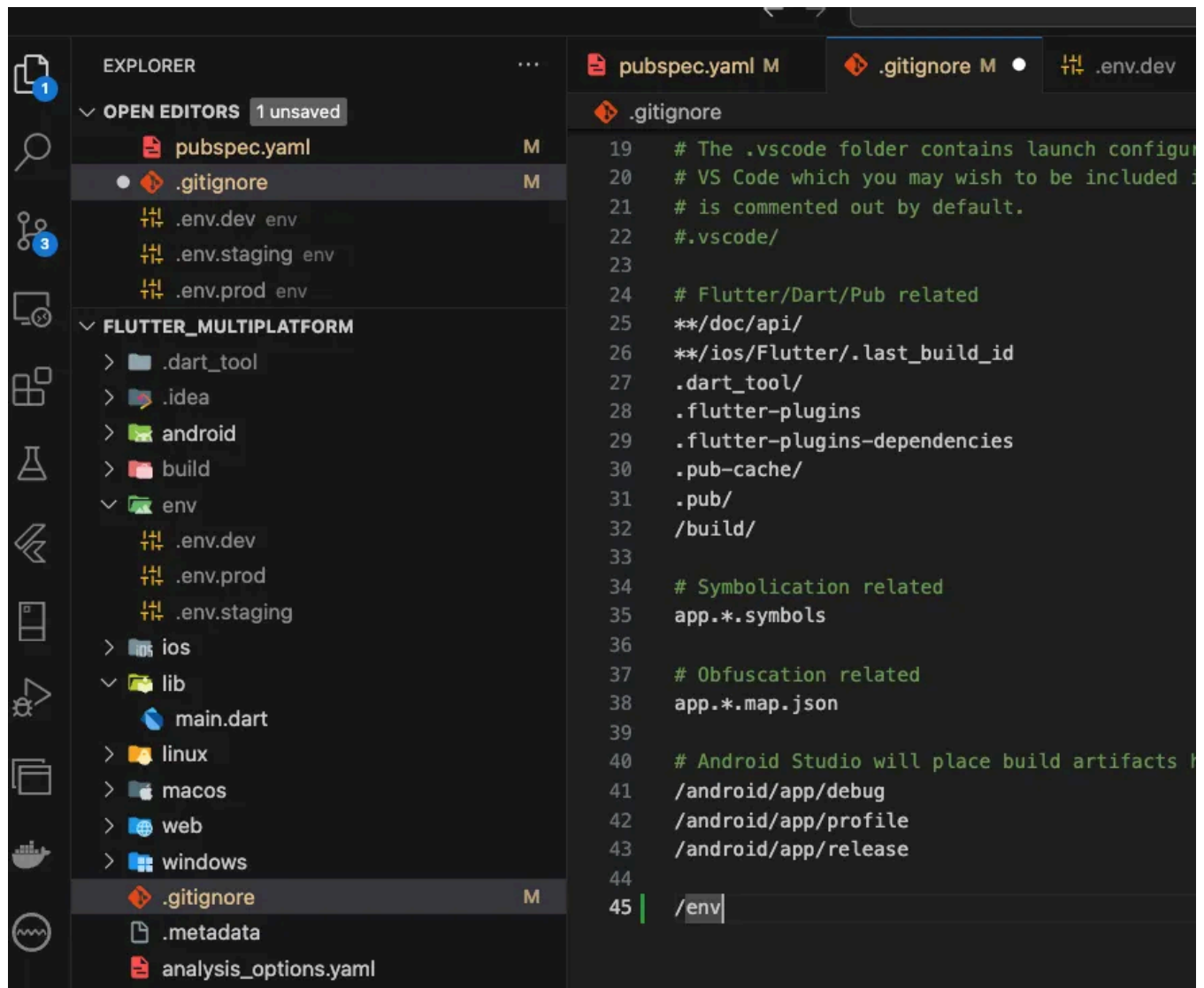
The next step is to add the `flutter_dotenv` package to your `pubspec.yaml` file:

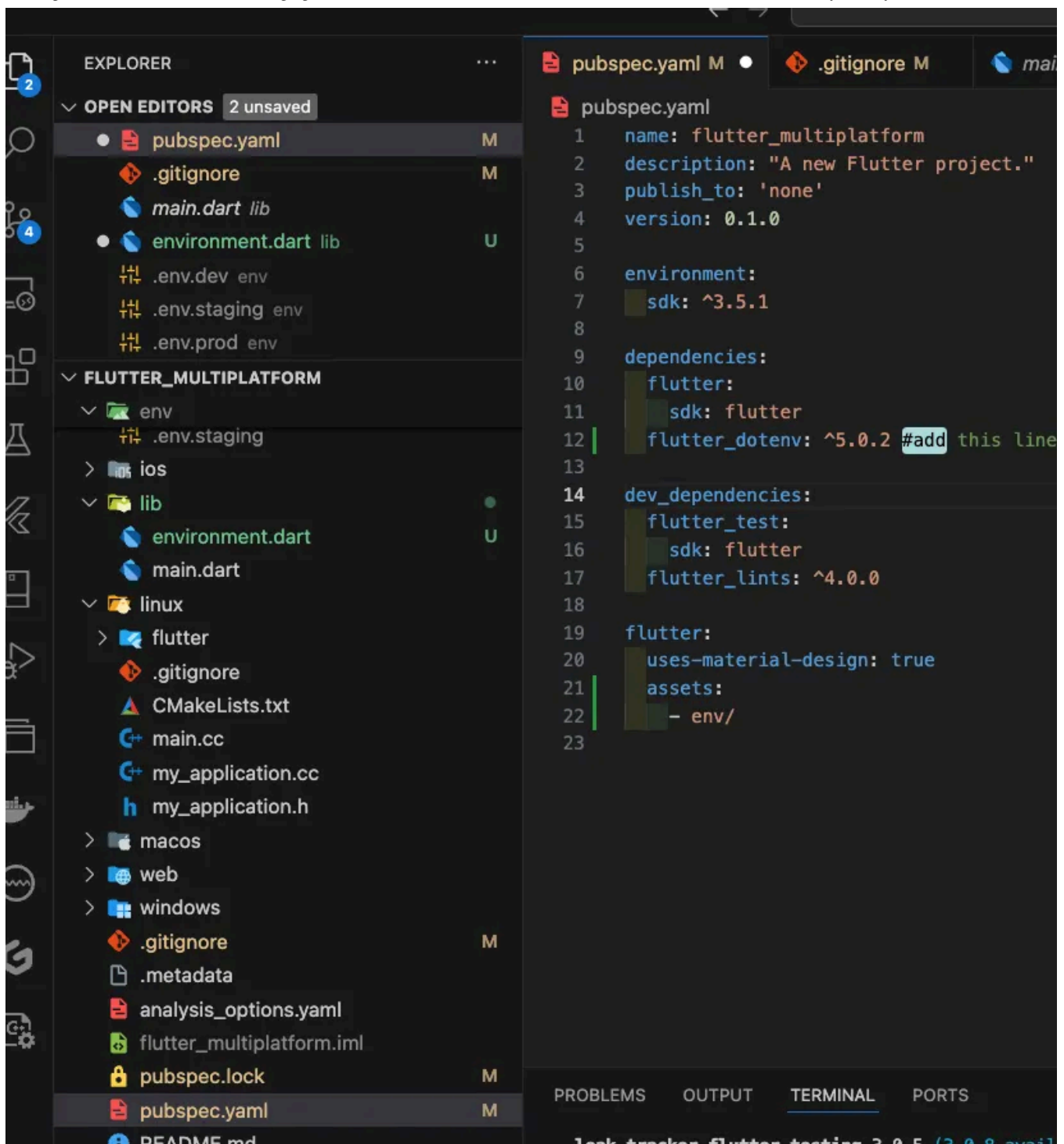


Now, we will create `.env` files in the `env` directory of your project directory. These files will store your environment variables depending on your flavors. Let's add `.env.dev`, `.env.prod`, and `.env.staging` files, and add a sample environment variable (let's say `APP_ENV` and assign their corresponding environment name).



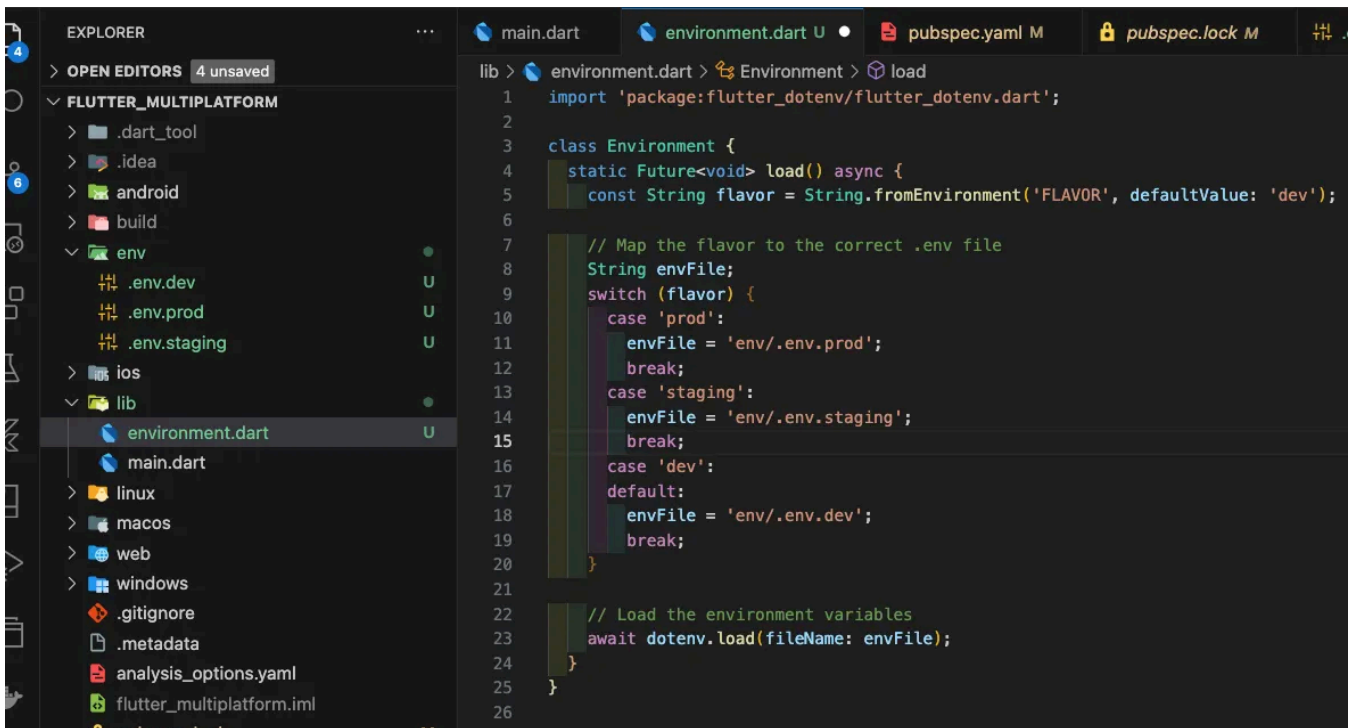
The next step is to add `/env` to the `.gitignore` file and also add `env` to `assets` in the `pubspec.yaml` file.



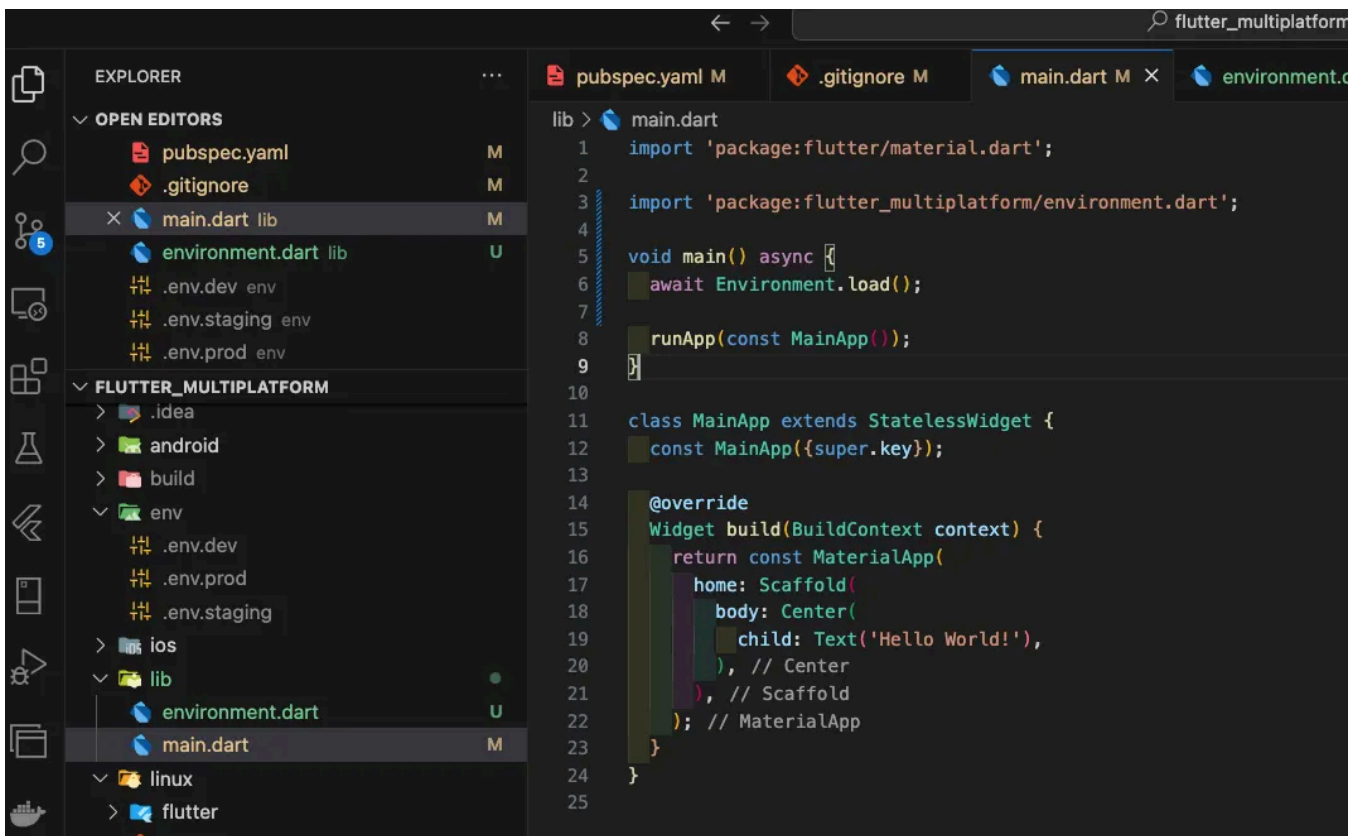


Load the Environment Variables

In your `lib` directory, create a `environment.dart` file to load the appropriate `.env` file based on the flavor specified at build time.



At this point, we need to load the `environment.dart` in your `main.dart` file.

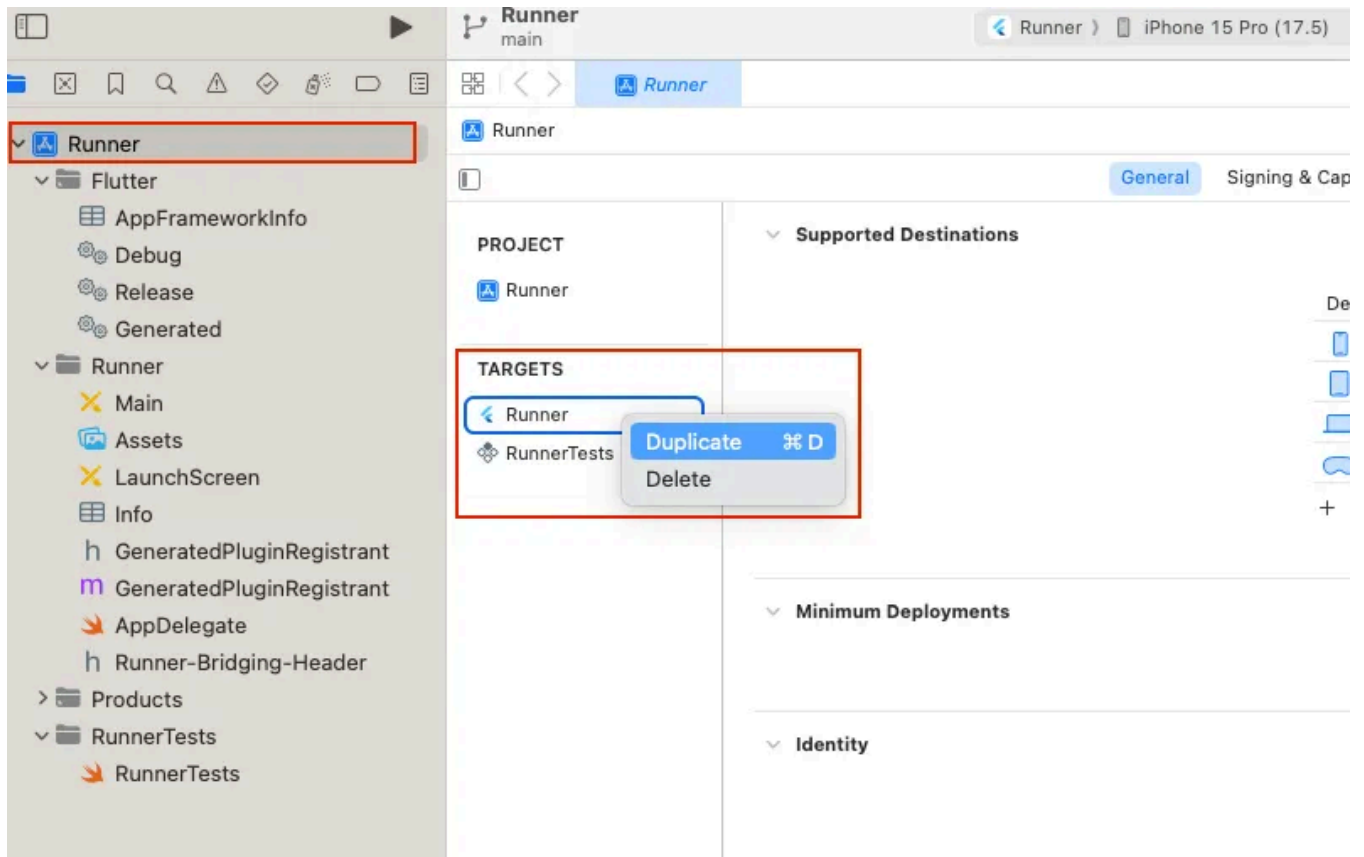


Create Flavors for iOS

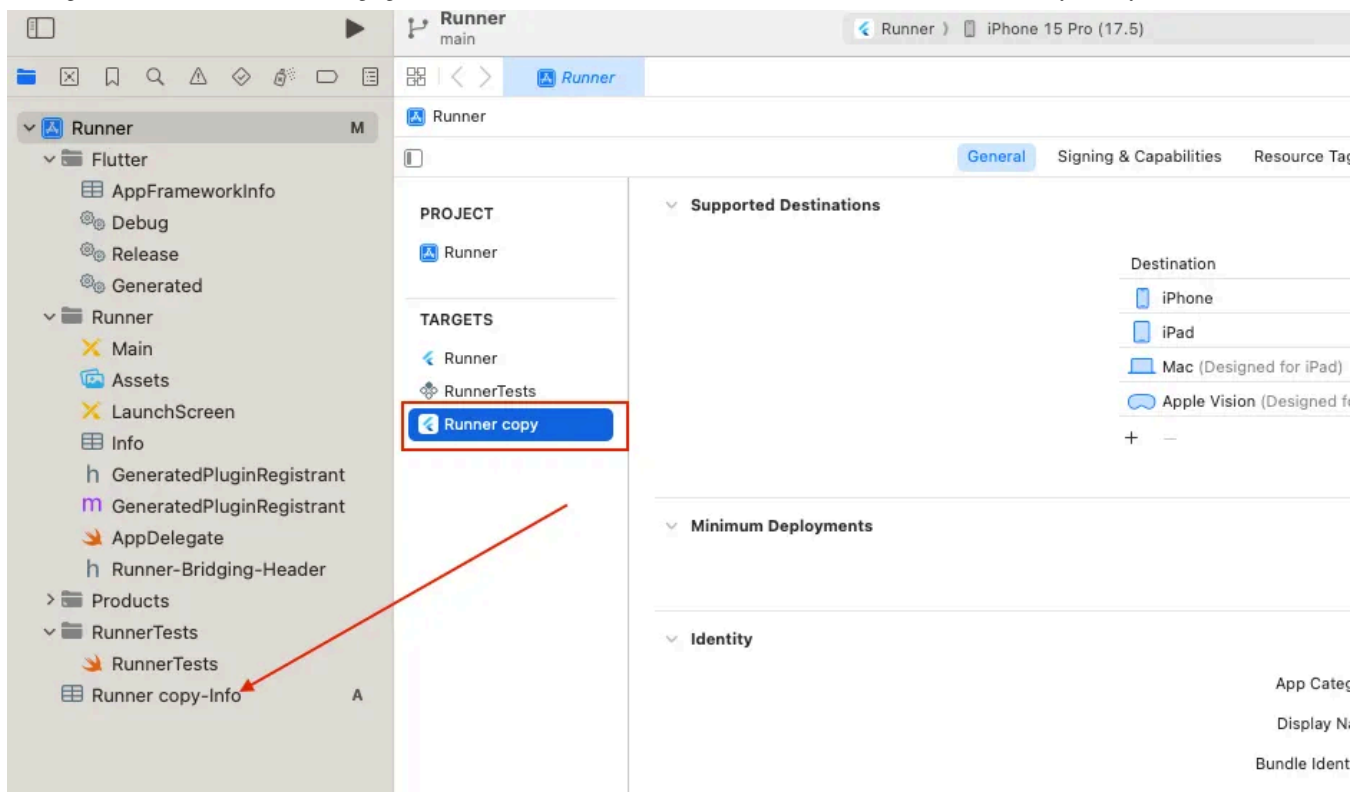
Setting up flavors for iOS in Flutter requires configuring Xcode to recognize and build different flavors (or schemes). Here's how you can do it:

Open your Flutter project in Xcode by navigating to the `ios` directory and opening the `Runner.xcworkspace` file.

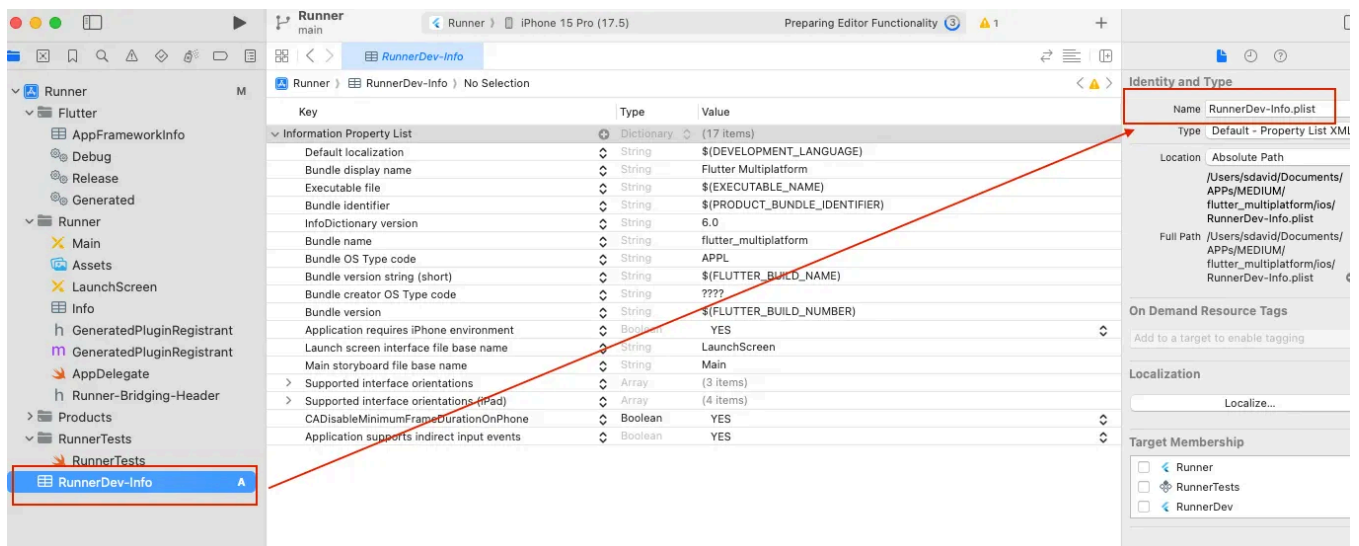
Click on the `Runner` in the root directory and then right-click on the `Runner` in the targets for duplication.



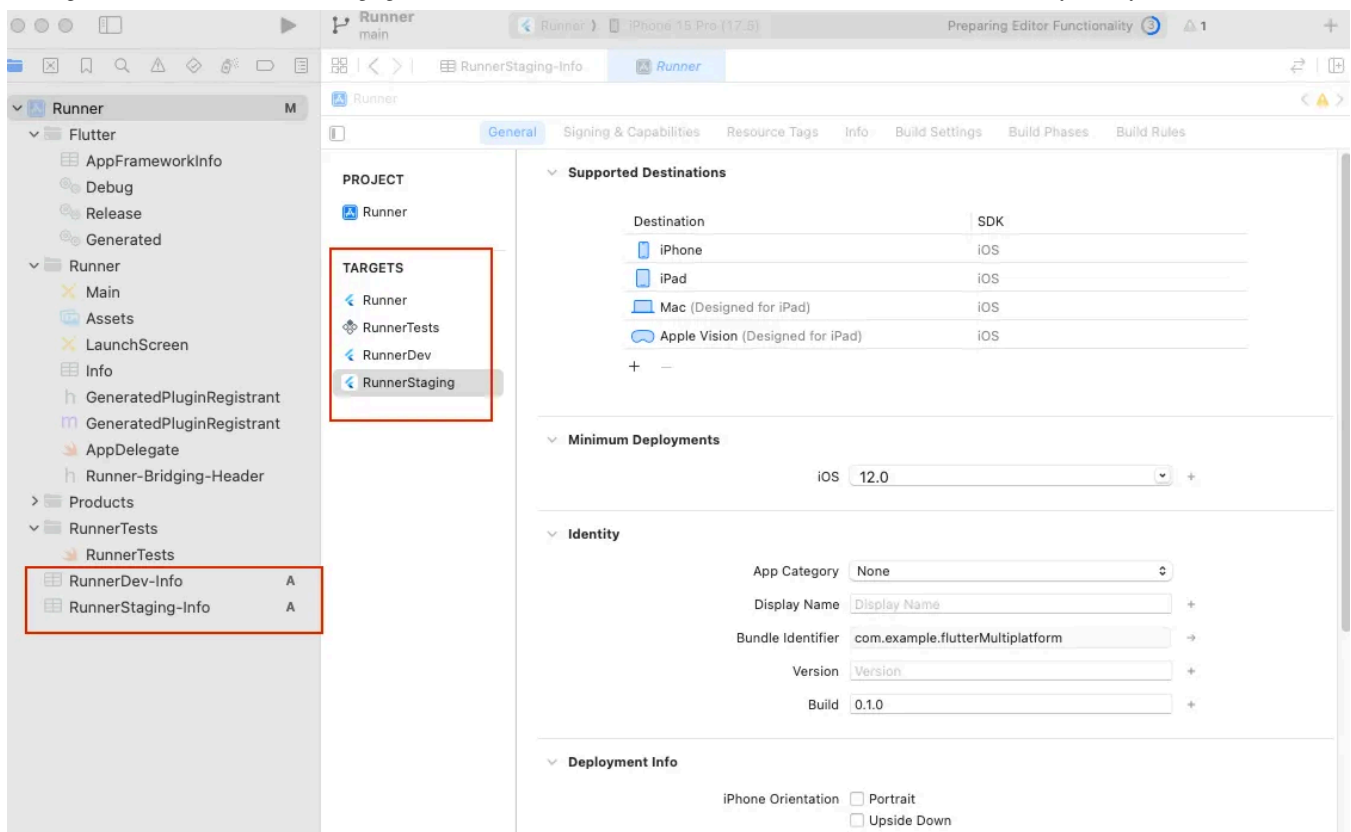
You'll notice a `Runner` copy created, and the goal is to rename this with your flavor suffix. You'll also notice that `Runner` copy-Info file is also created as shown in the image below.



Rename `Runner copy` (with a double click) to `RunnerDev`, and `Runner copy-Info` to `RunnerDev-Info.plist`. Repeat the same process for `RunnerStaging`. Also, rename the `Runner copy-Info2` to `RunnerStaging-Info.plist`.

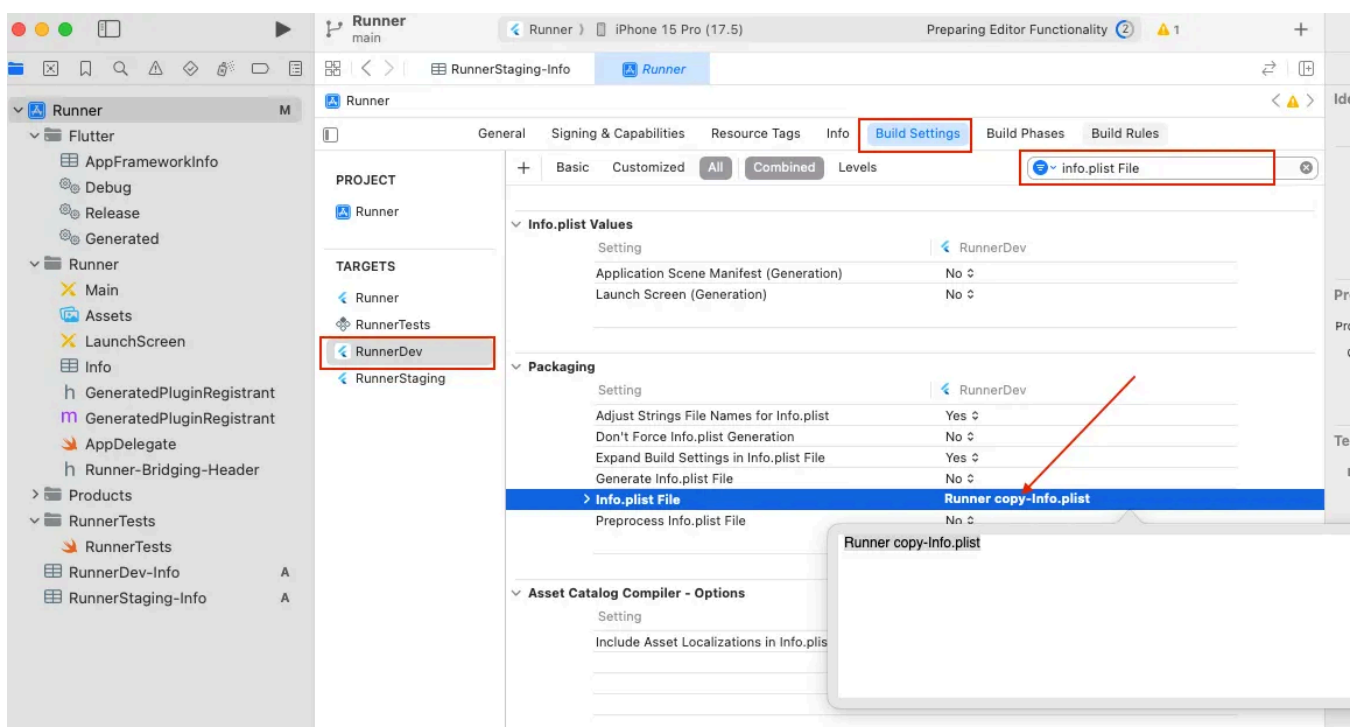


We should have the resulting files as shown below:

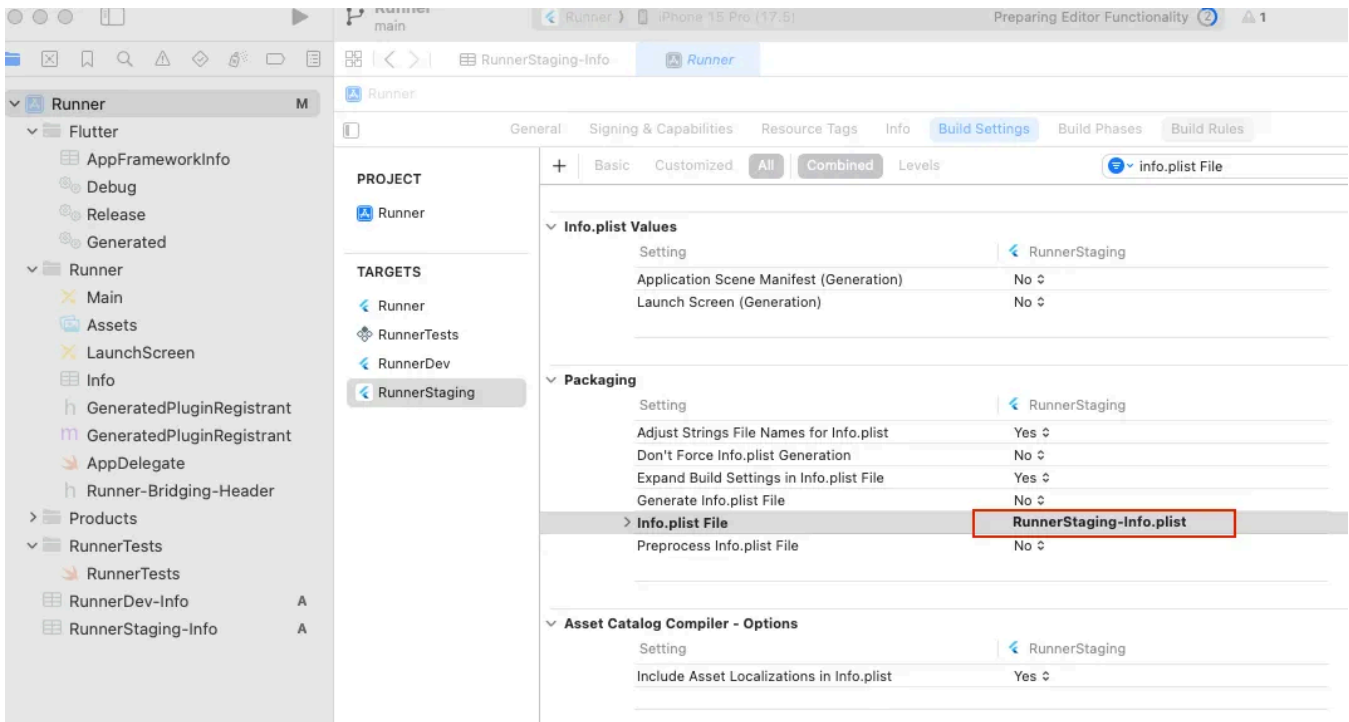


Assign plist to Targets

Click on the `RunnerDev` then go to the `Build Settings` tab, and search for `info.plist` file. Double-click on the value under the `Packaging` section on the right side next to `Info.plist` File.

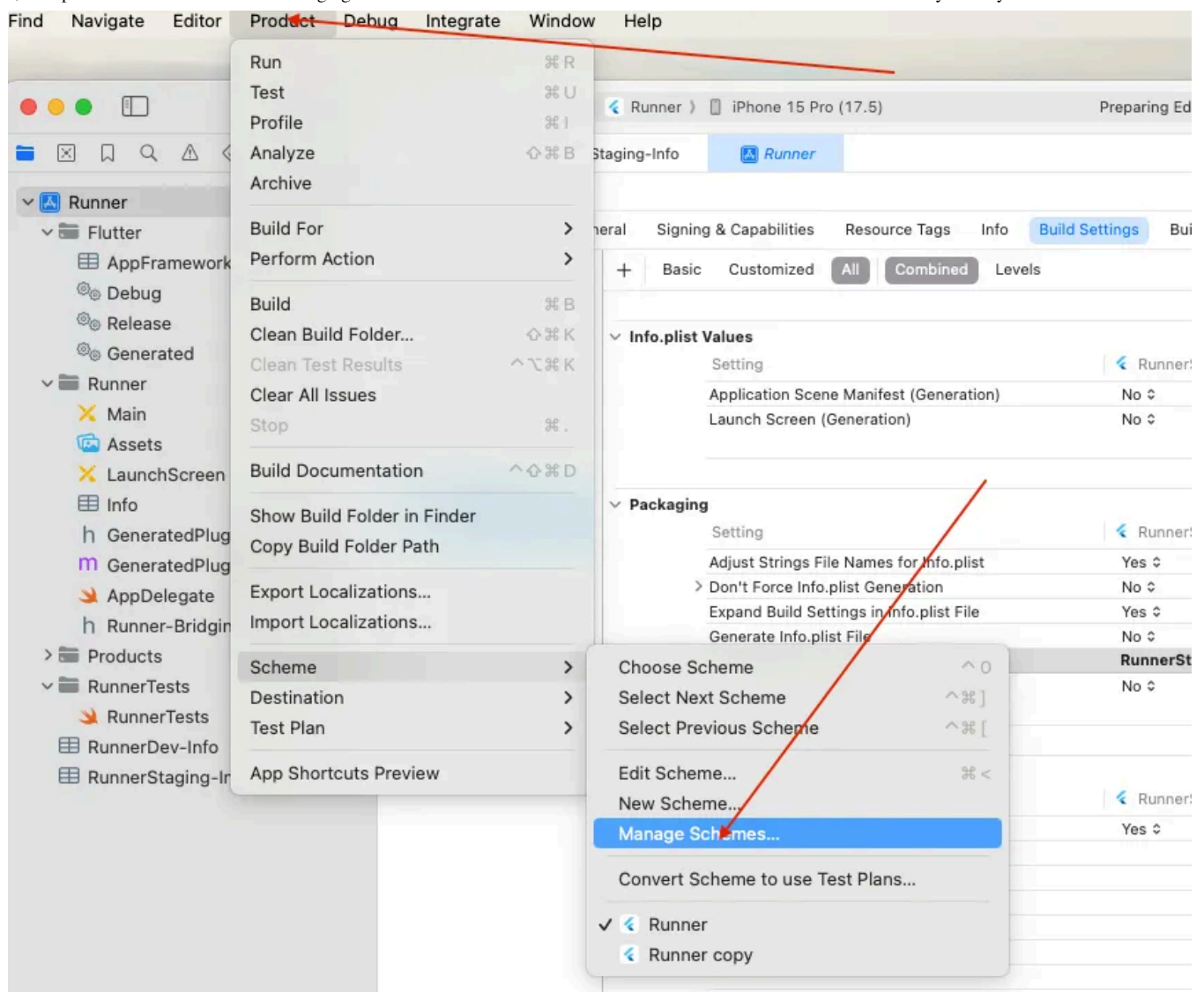


Rename Runner copy-info.plist to RunnerDev-info .Repeat the same step for RunnerStaging .



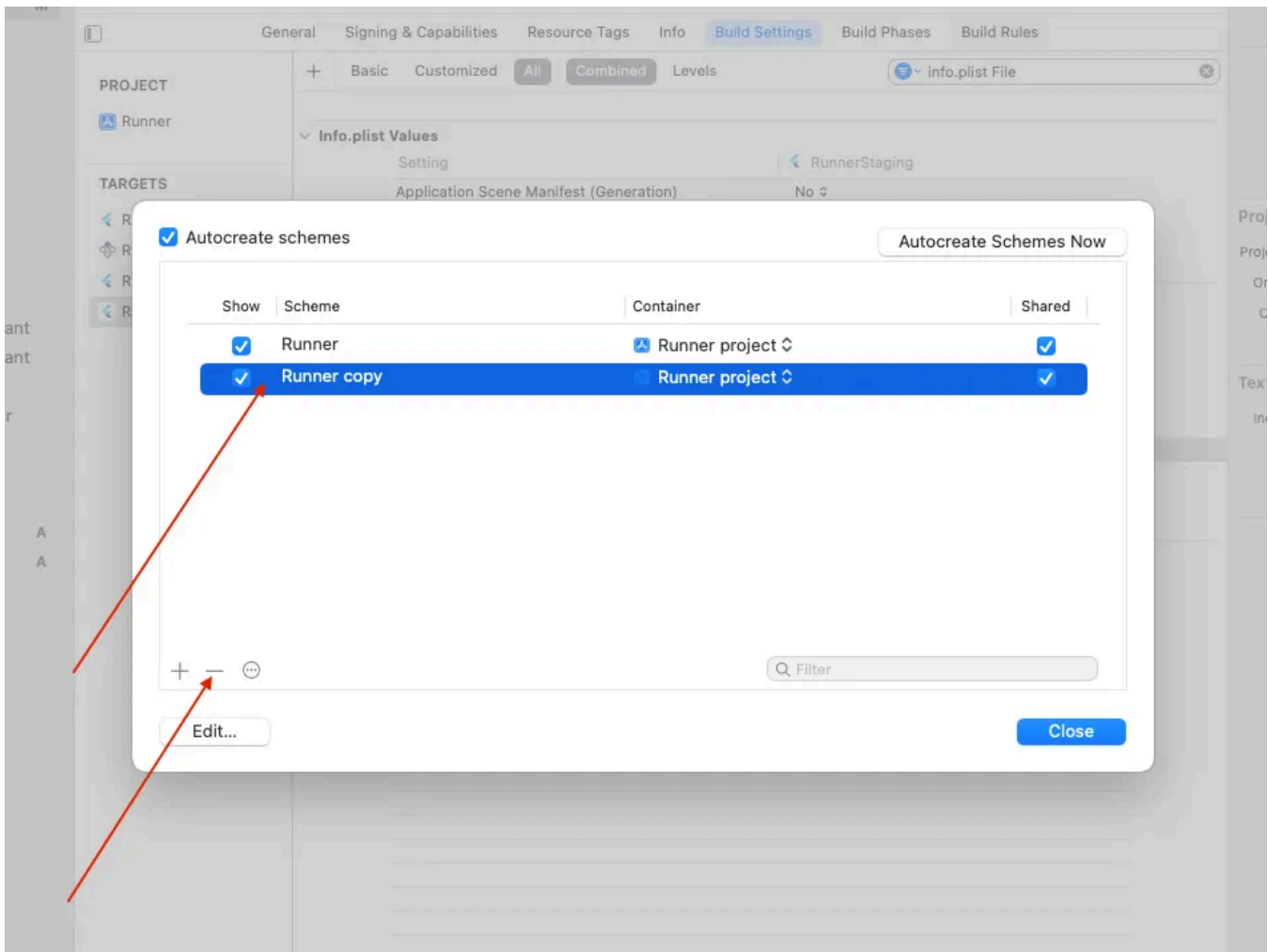
Create Schemes for Each Flavor

Navigate to manage schemes as shown in the picture below:

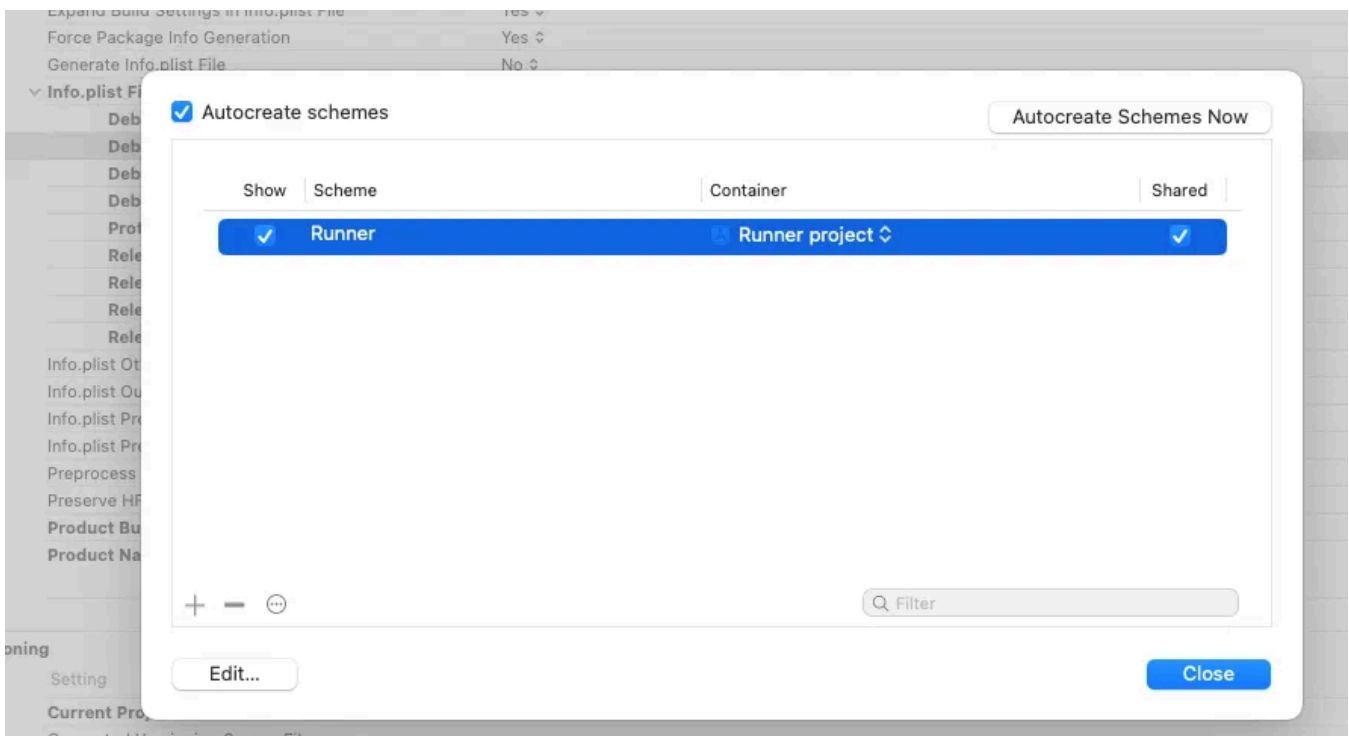


Duplicate the current scheme for each flavor:

If you have other schemes apart from the `Runner`, your life would be easier to remove them first before continuing. In my case, I'll remove `Runner copy` by selecting it and clicking on the `-` button. Then I'll duplicate the `Runner` and rename it `RunnerDev` and another as `RunnerStaging`.

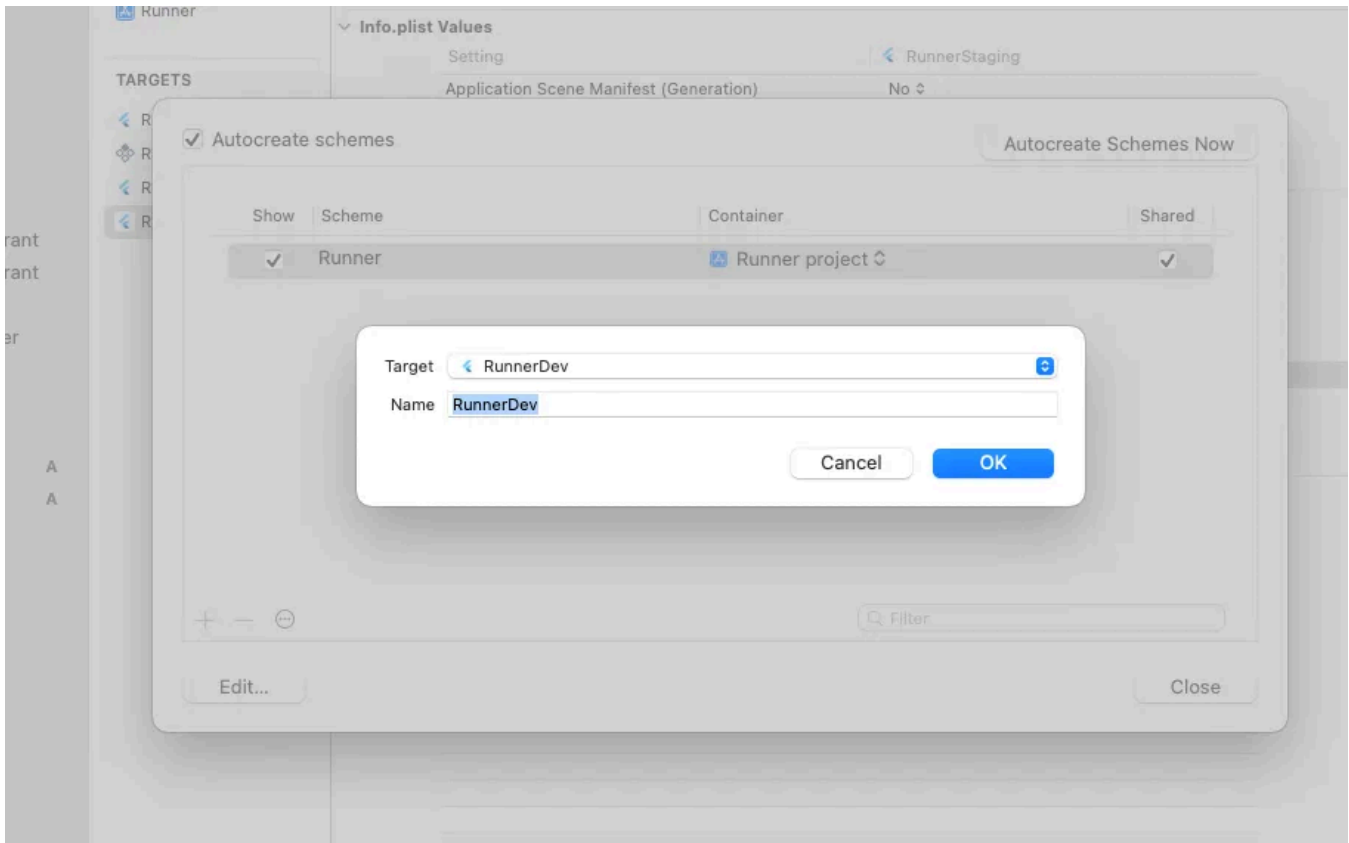


Result:

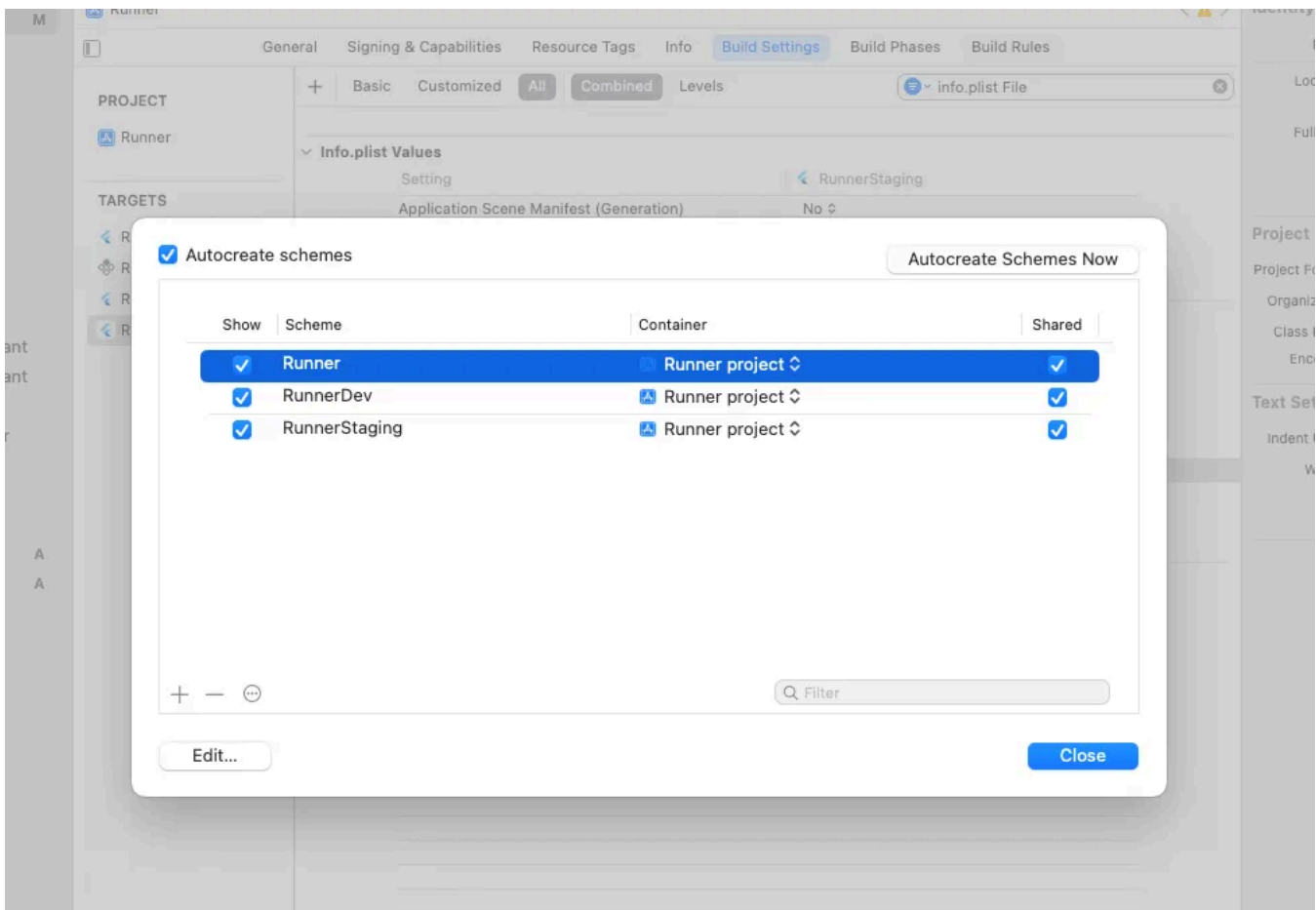


- Click on Runner, and then + button > Duplicate Scheme > Rename to **RunnerDev** .

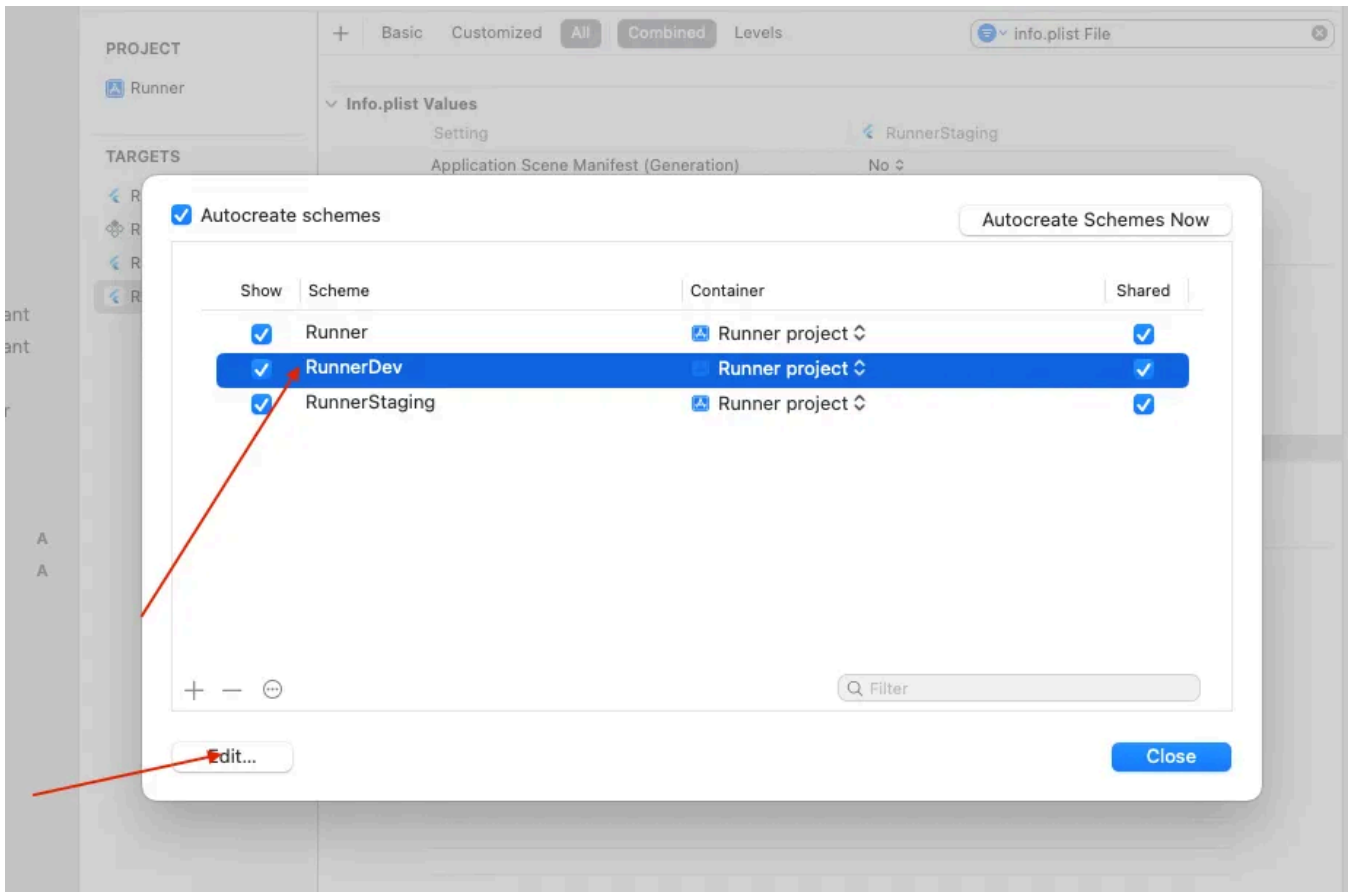
- Repeat this for other flavors - RunnerStaging).



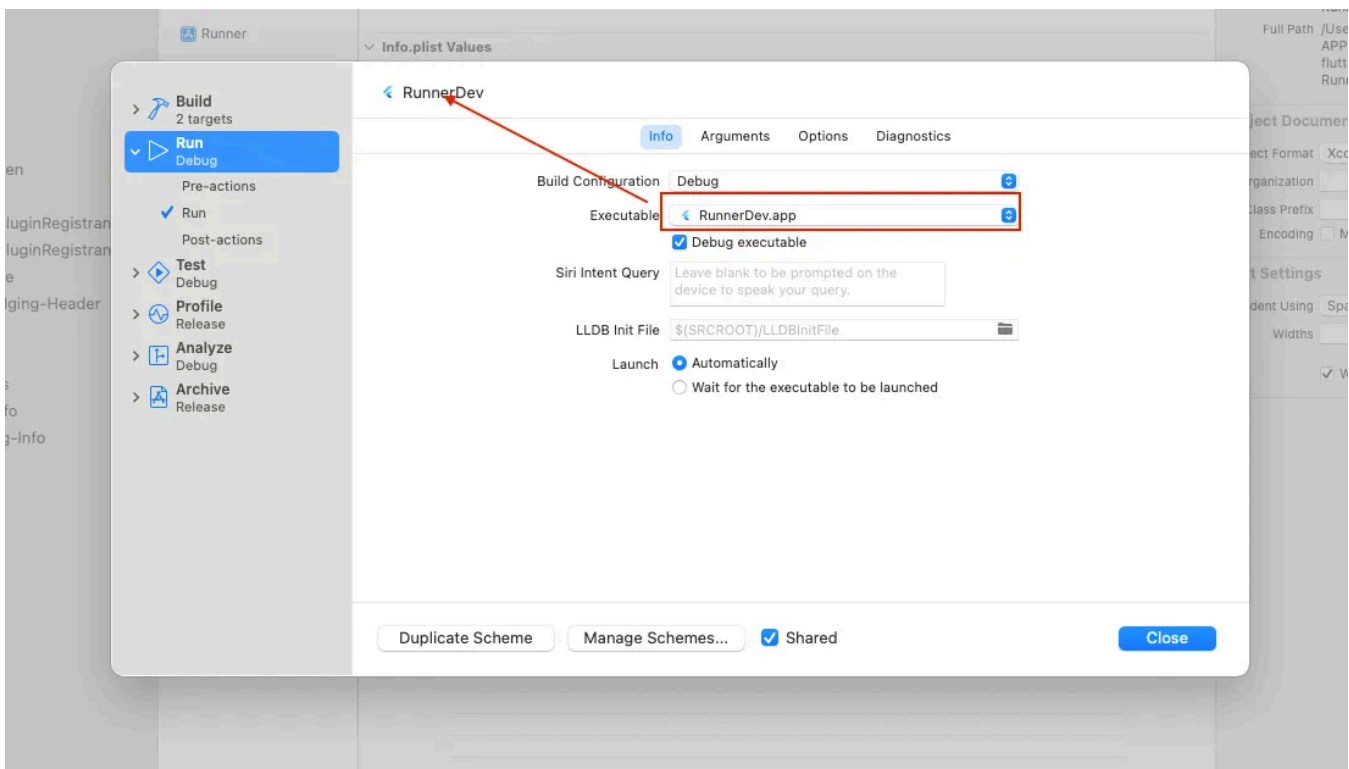
Resulting outcome:

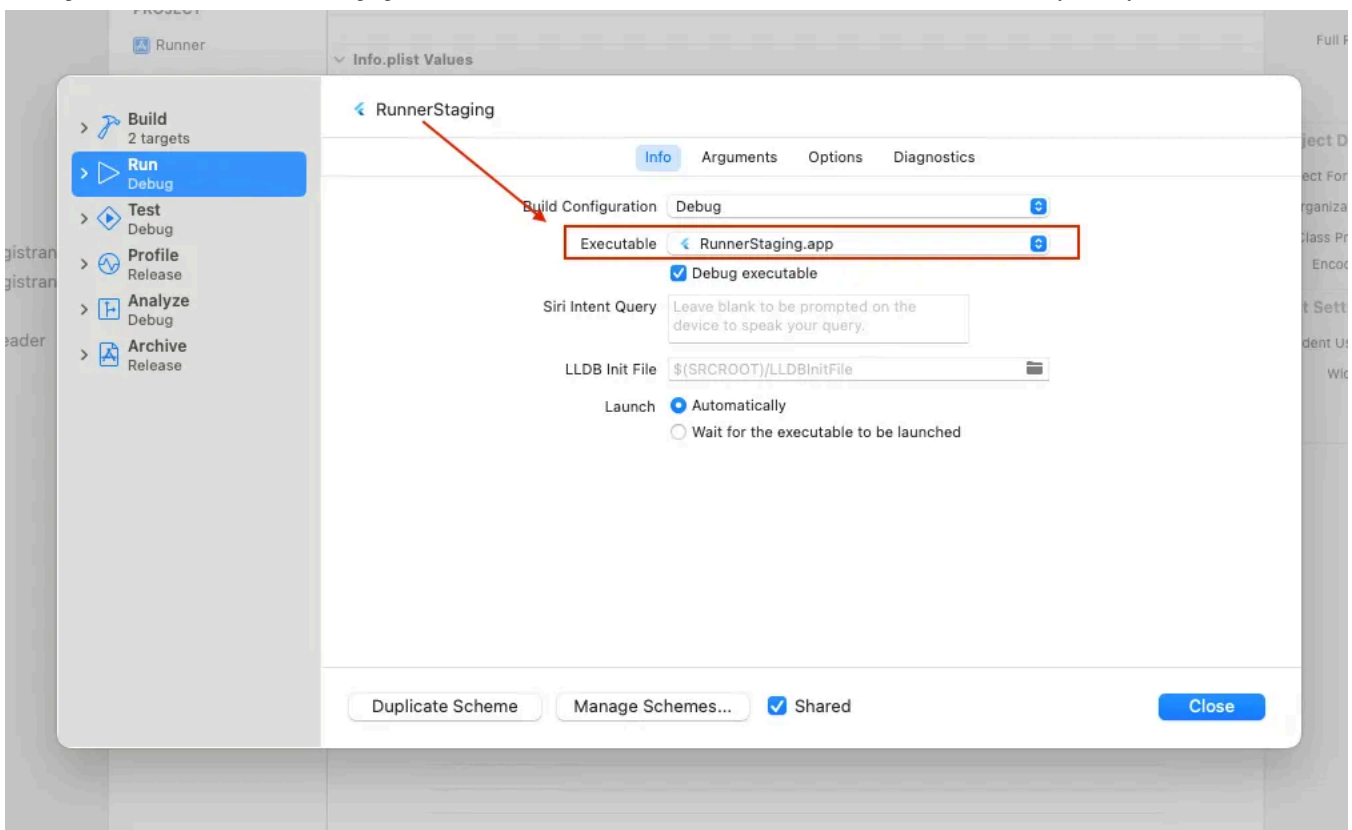


Select the scheme and click “Edit” as shown below



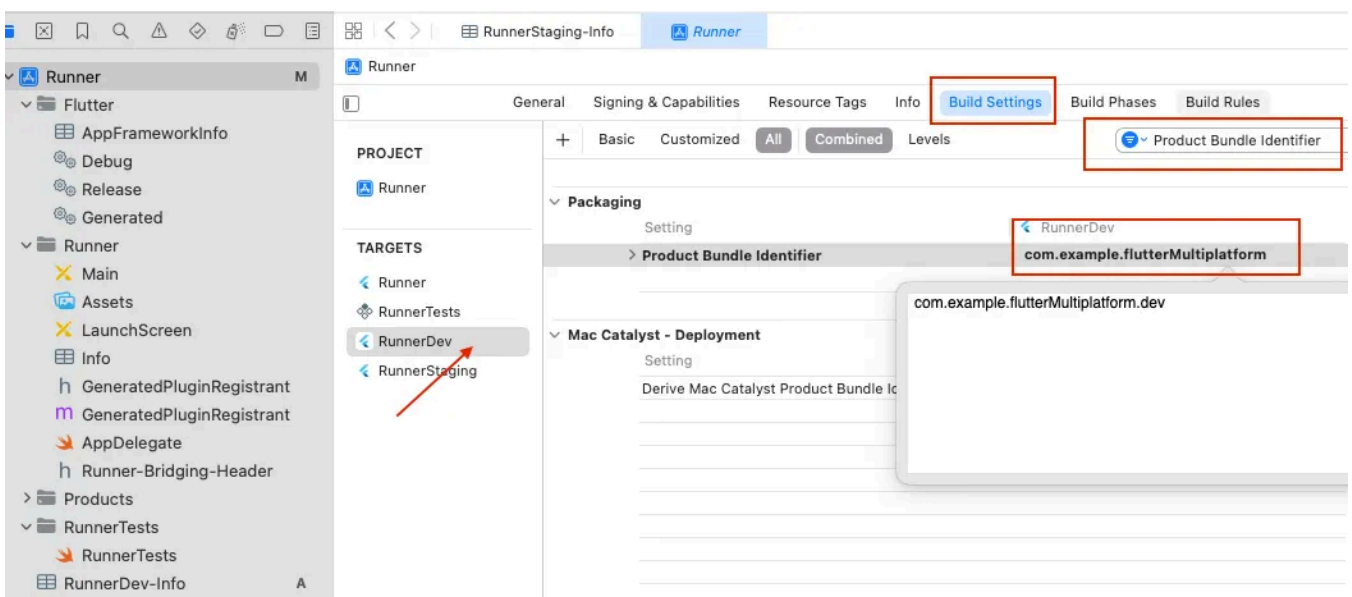
- In the “Run” section, ensure that RunnerDev is assigned as the executable for RunnerDev as shown below:



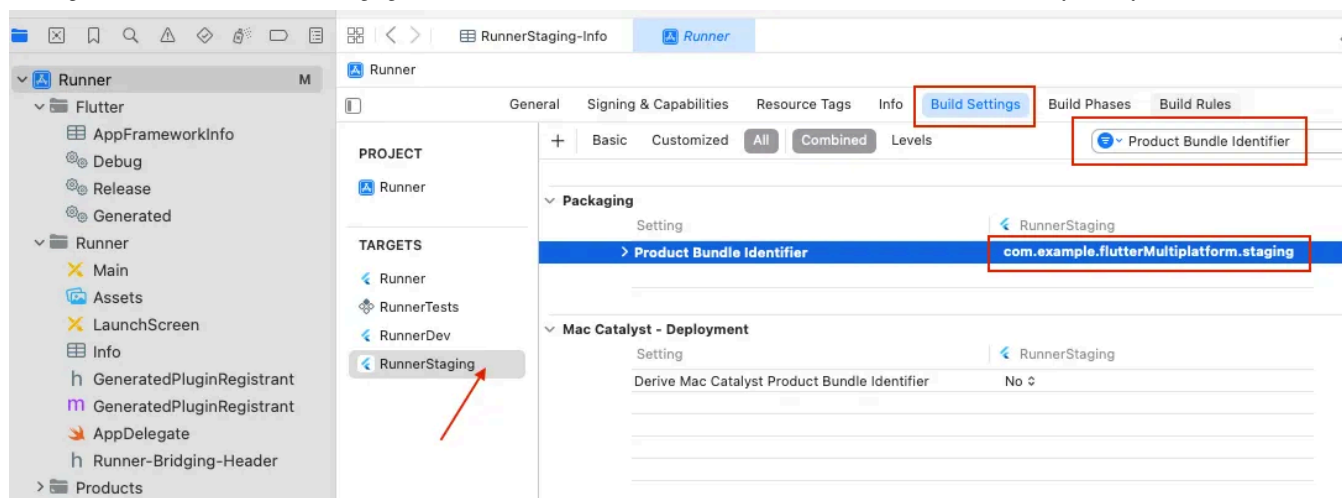


Bundle Identifier for each environment App

In order to assign Bundle Identifier to each of your targets, select the project, then select the target you want to name. Click on the **Build Settings** tab then search **Product Bundle Identifier**. Double-click on the value and add the flavor suffix (**dev** and **staging**)



Repeat the same for staging:



Assign the corresponding FLAVOR to each Runner

Get Sunday David's stories in your inbox

Join Medium for free to get updates from this writer.

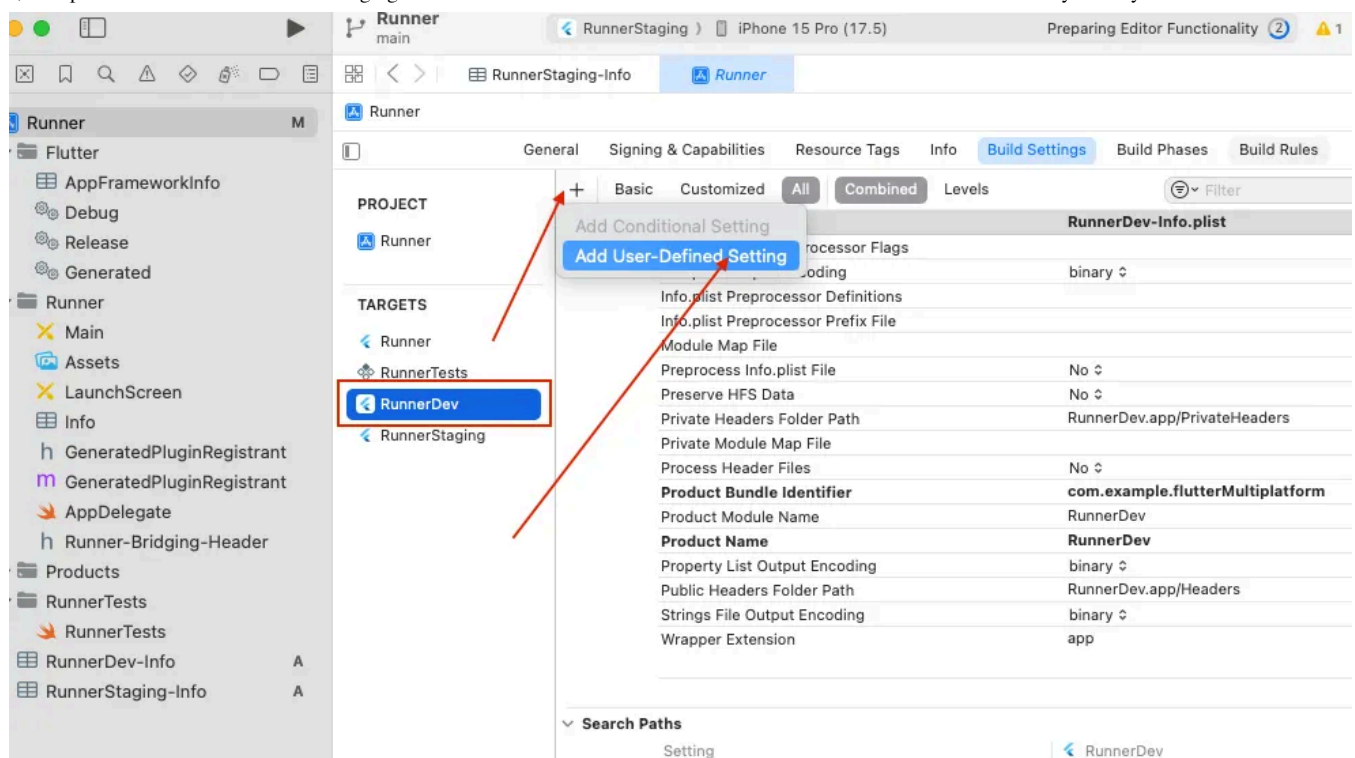
Enter your email

Subscribe

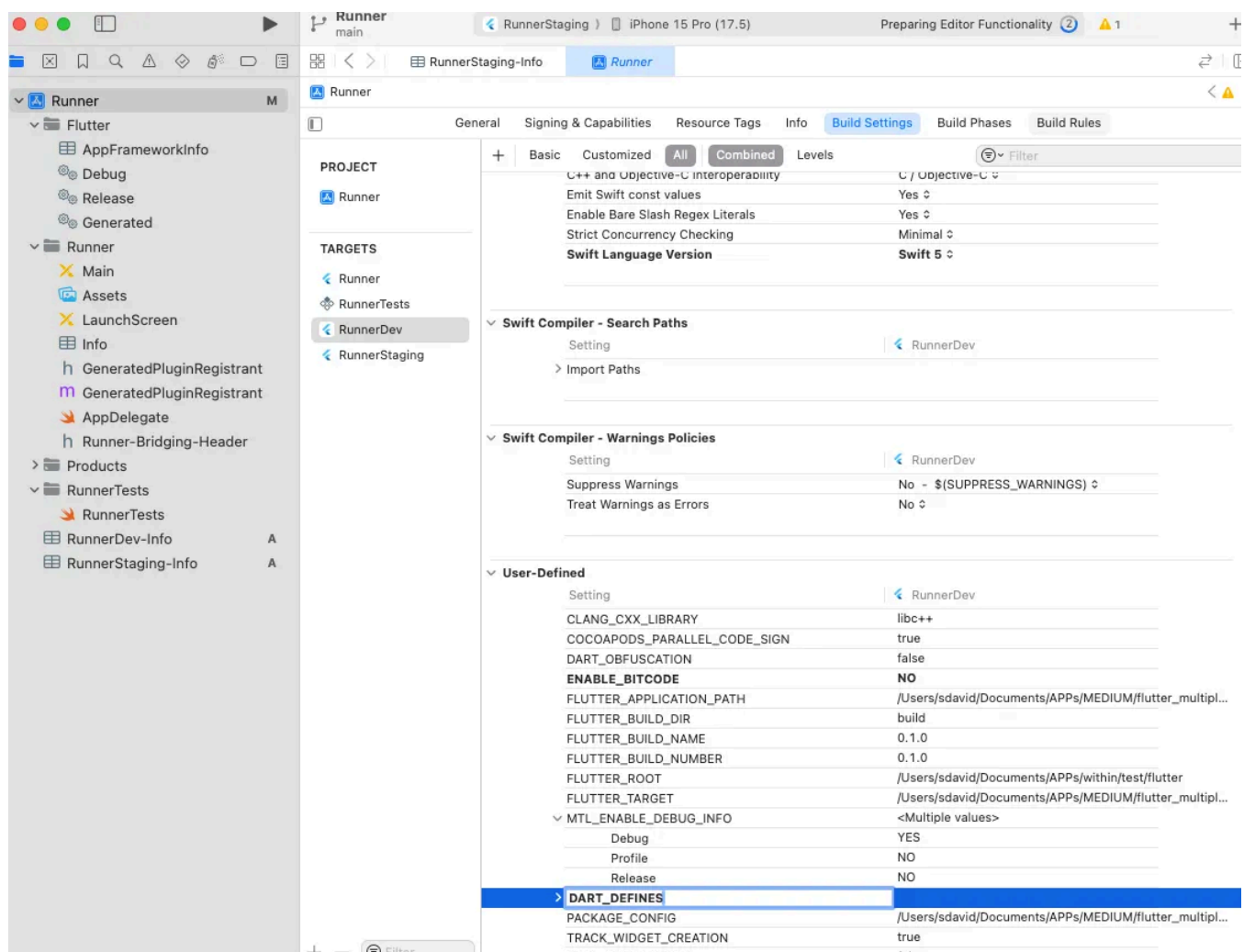
☐ Remember me for faster sign in

The next step is to assign the corresponding FLAVOR to each runner using the user-defined settings.

Click on `RunnerDev` target, and then click on the `+` button to select `Add User-Defined Setting`.



Enter `DART_DEFINES` in the textbox that opened.

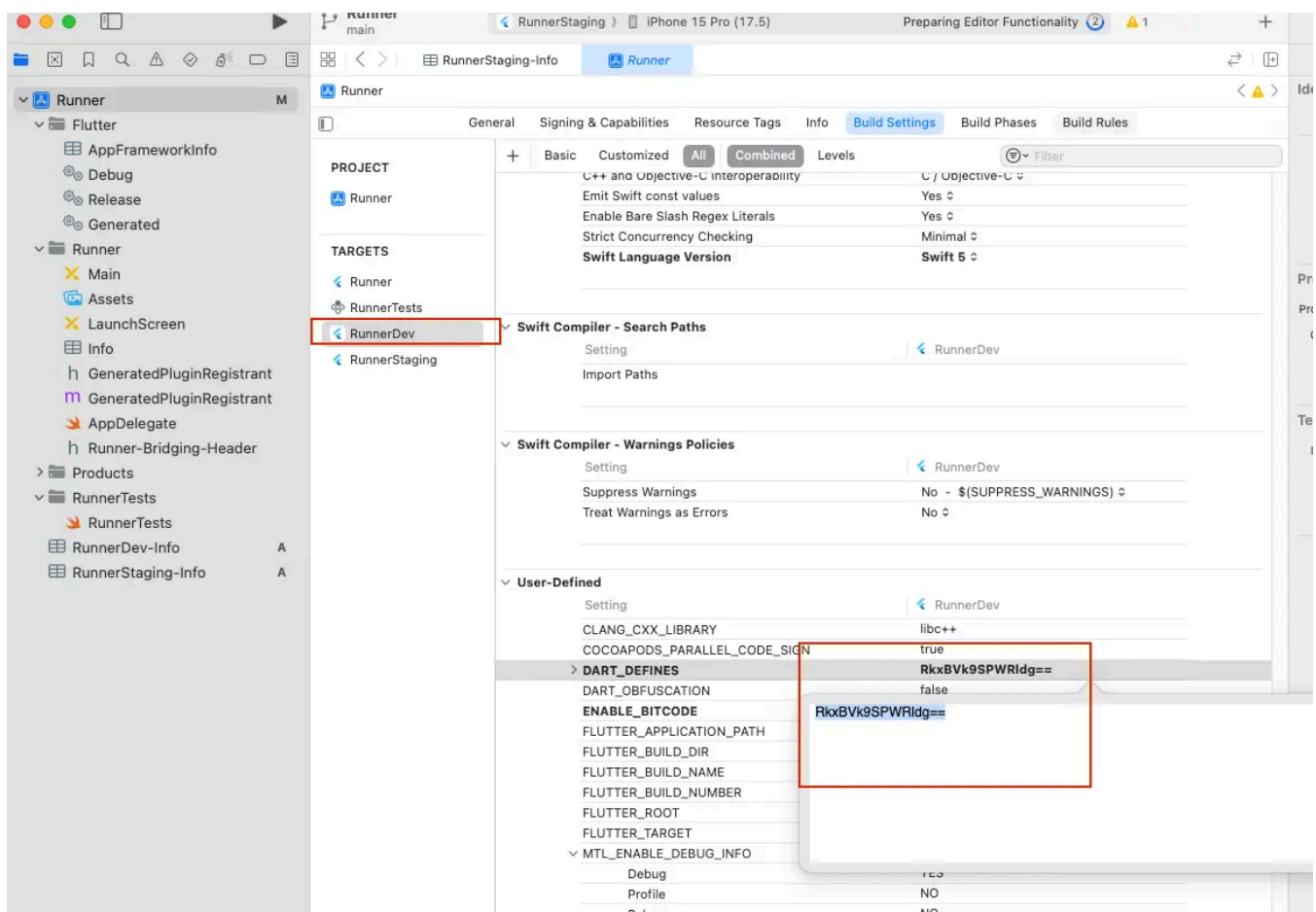


Now, we need to assign `base64` encoded string to this variable. Open your terminal, and run the following code for each of the `flavors`, that is, `dev`, `staging`, and `prod`.

```
echo -n "FLAVOR=dev" | base64
```

This will produce `RkxBVk9SPWRldg==`

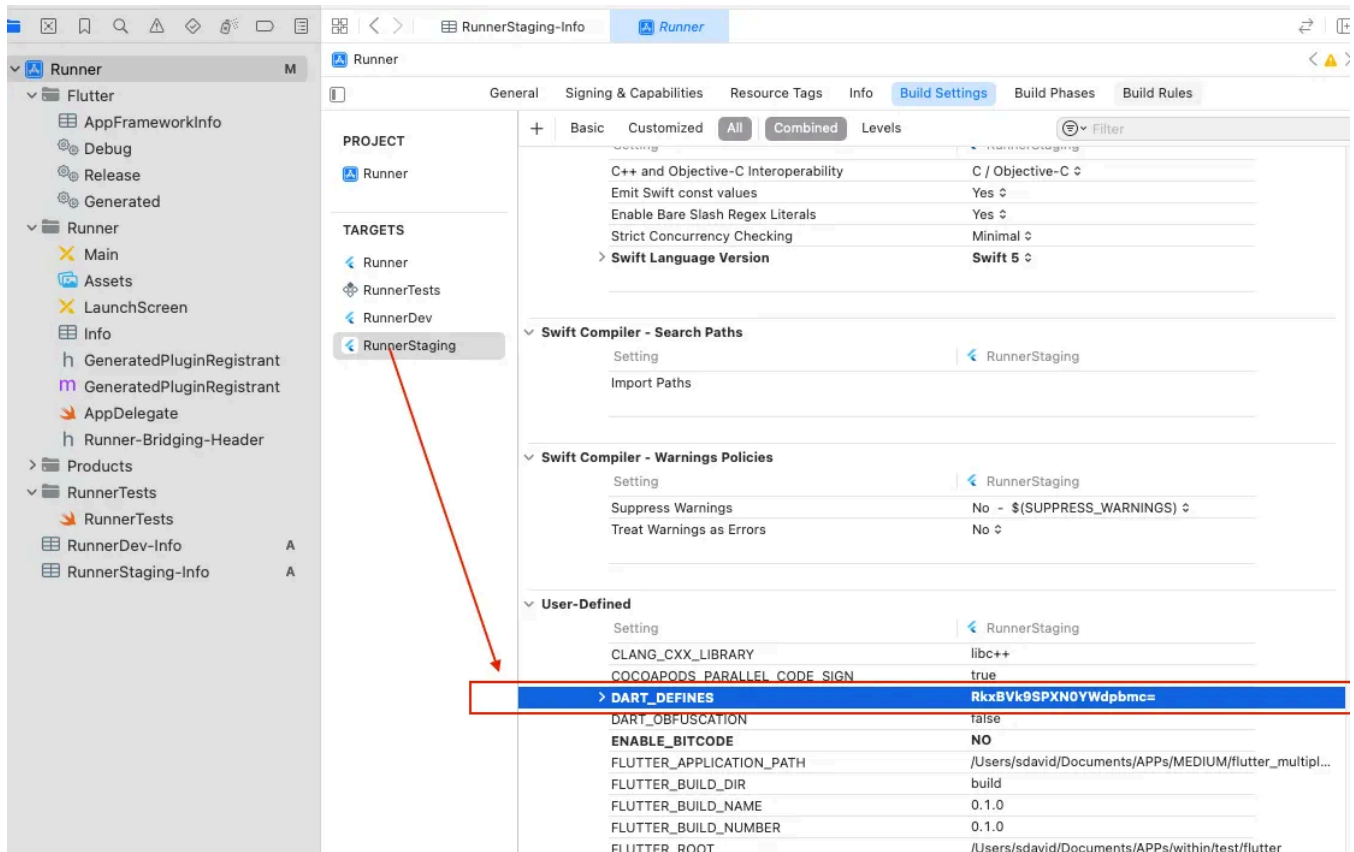
Assign this value to the `DART_DEFINES` by double-clicking on the right side, and paste this value.



For staging:

```
echo -n "FLAVOR=staging" | base64
```

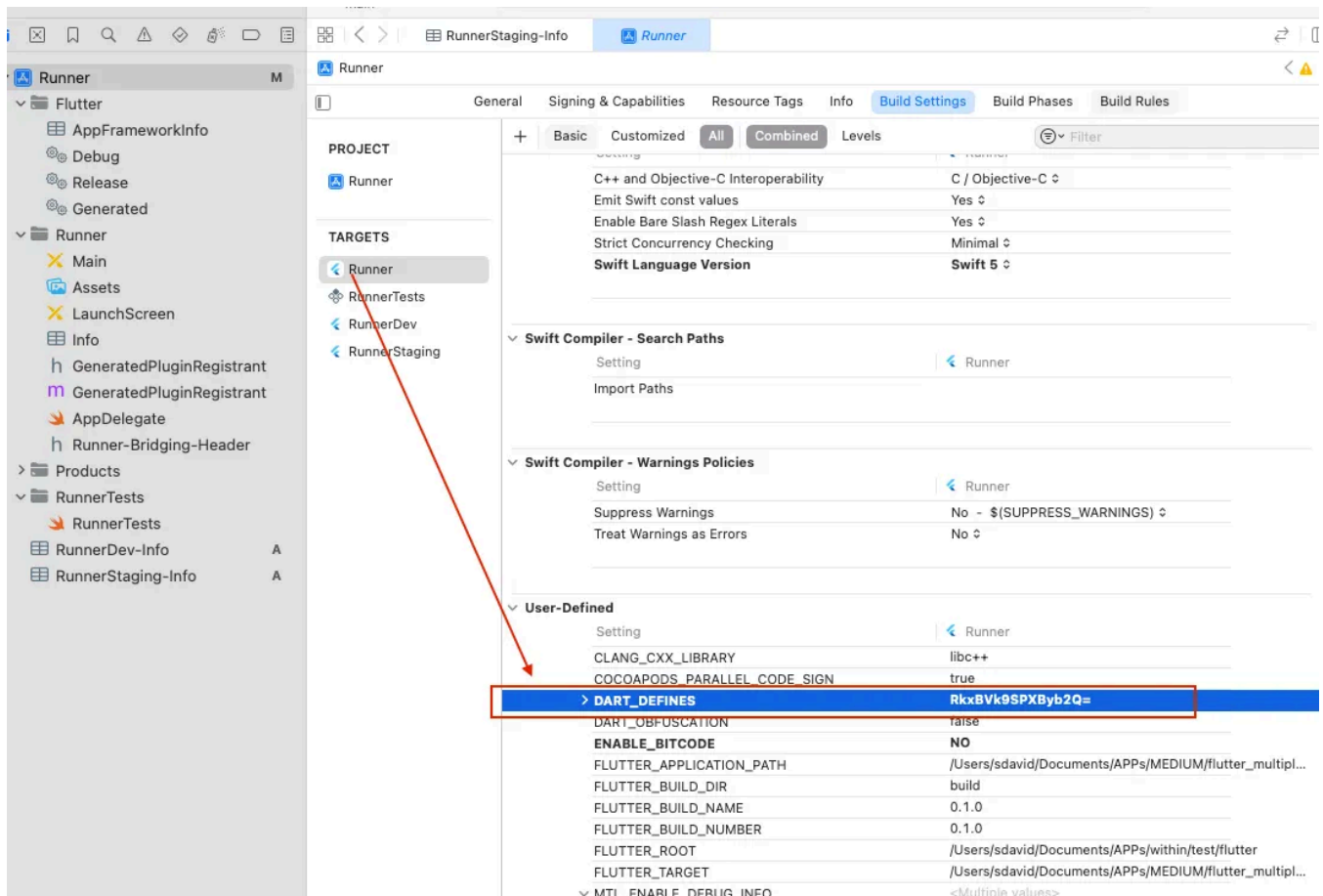
This will produce `RkxBVk9SPXNOYWdpbmc=`



For prod:

```
echo -n "FLAVOR=prod" | base64
```

This will produce RkxBVb9SPXByb2Q=



If you need to pass more Runner specific values at build time, you can add the variables using:

```
echo -n "FLAVOR=prod,API_BASE_URL=https://api.example.com" | base64
```

The resulting string can then be assigned to the `DART_DEFINES`.

Create Flavors for Android

For Android, add this code to `android/app/build.gradle` file as shown

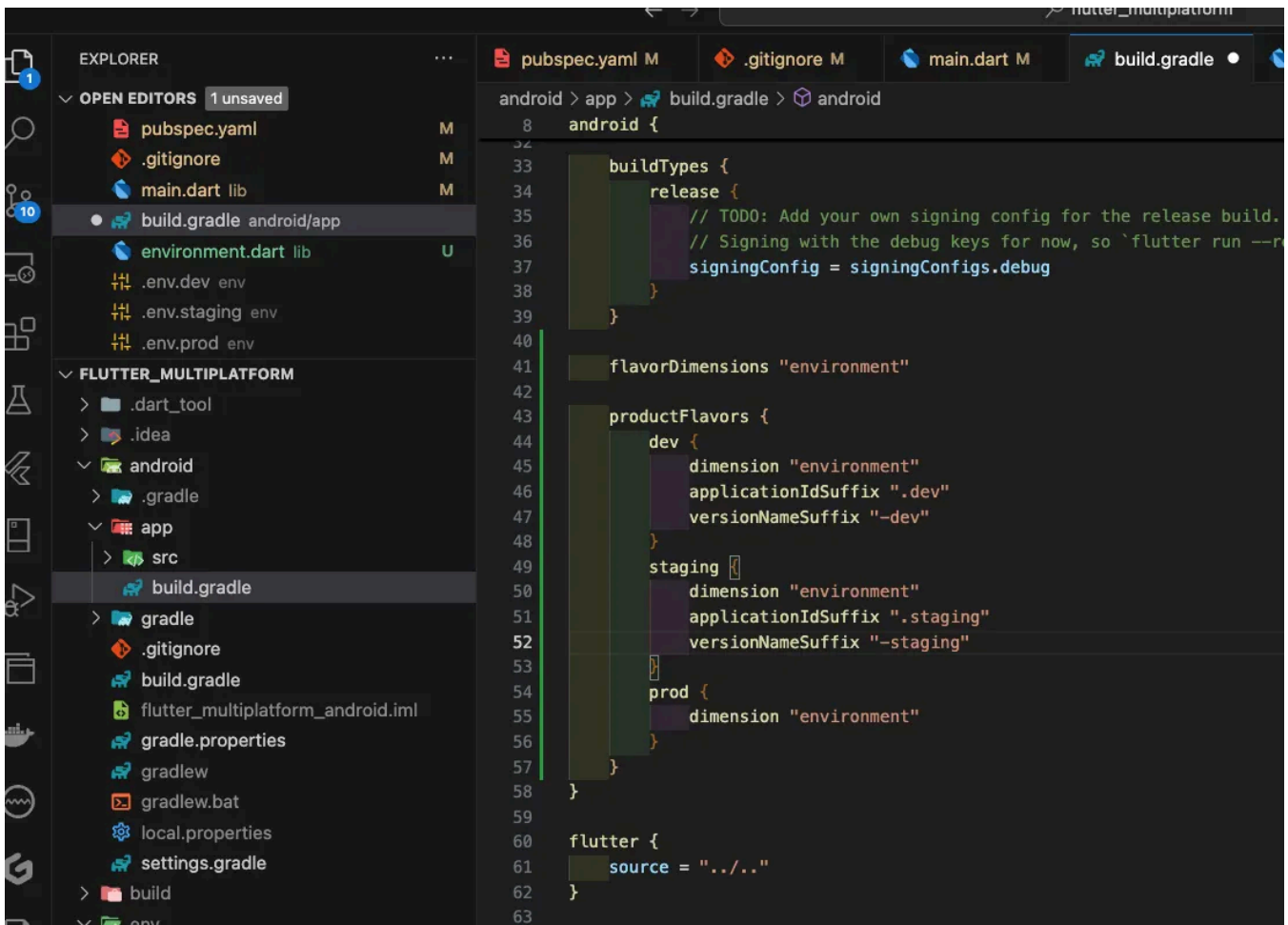
```
flavorDimensions "environment"

productFlavors {
    dev {
        dimension "environment"
        applicationIdSuffix ".dev"
        versionNameSuffix "-dev"
    }
    staging {
```

```

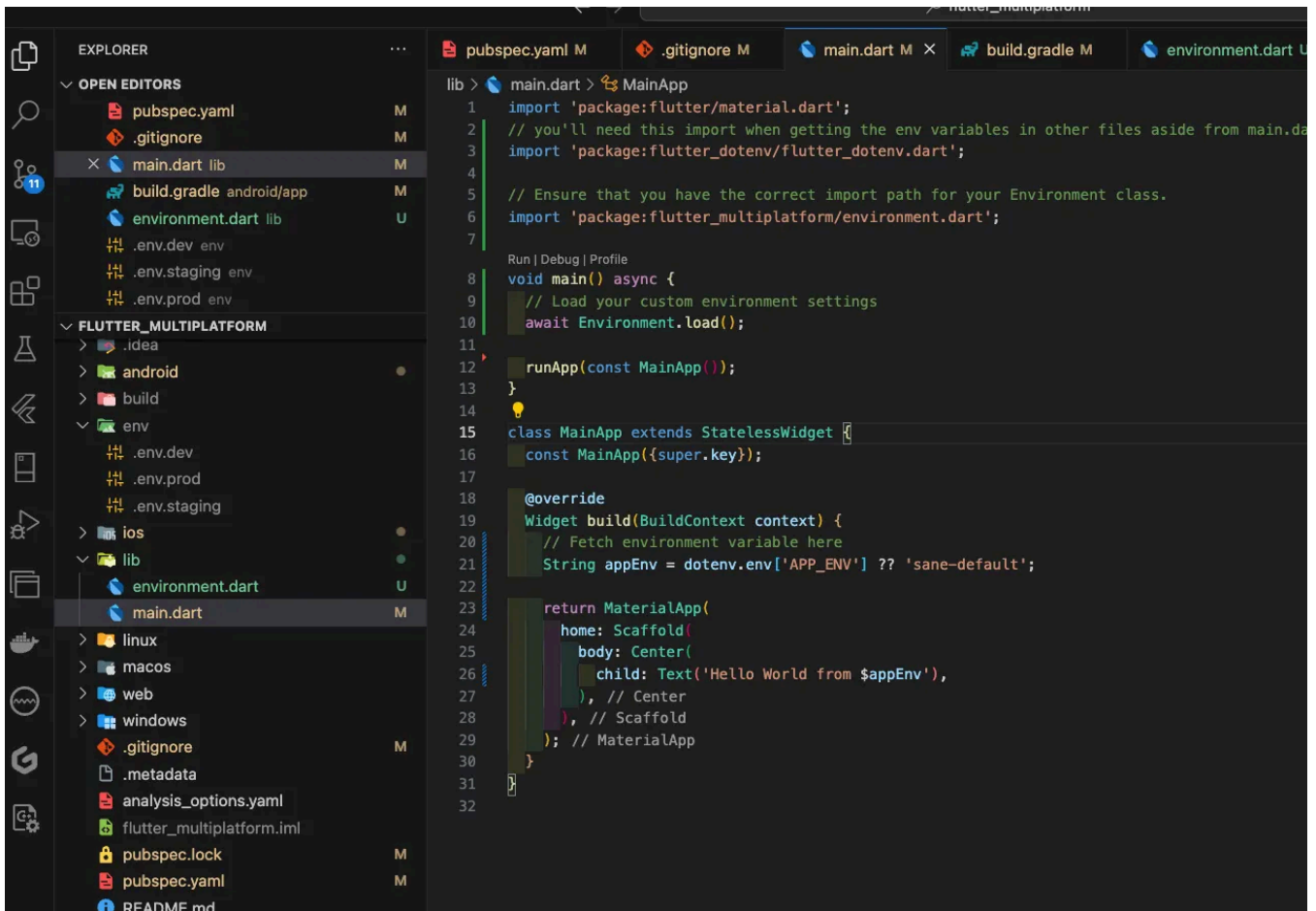
dimension "environment"
applicationIdSuffix ".staging"
versionNameSuffix "-staging"
}
prod {
  dimension "environment"
}
}

```

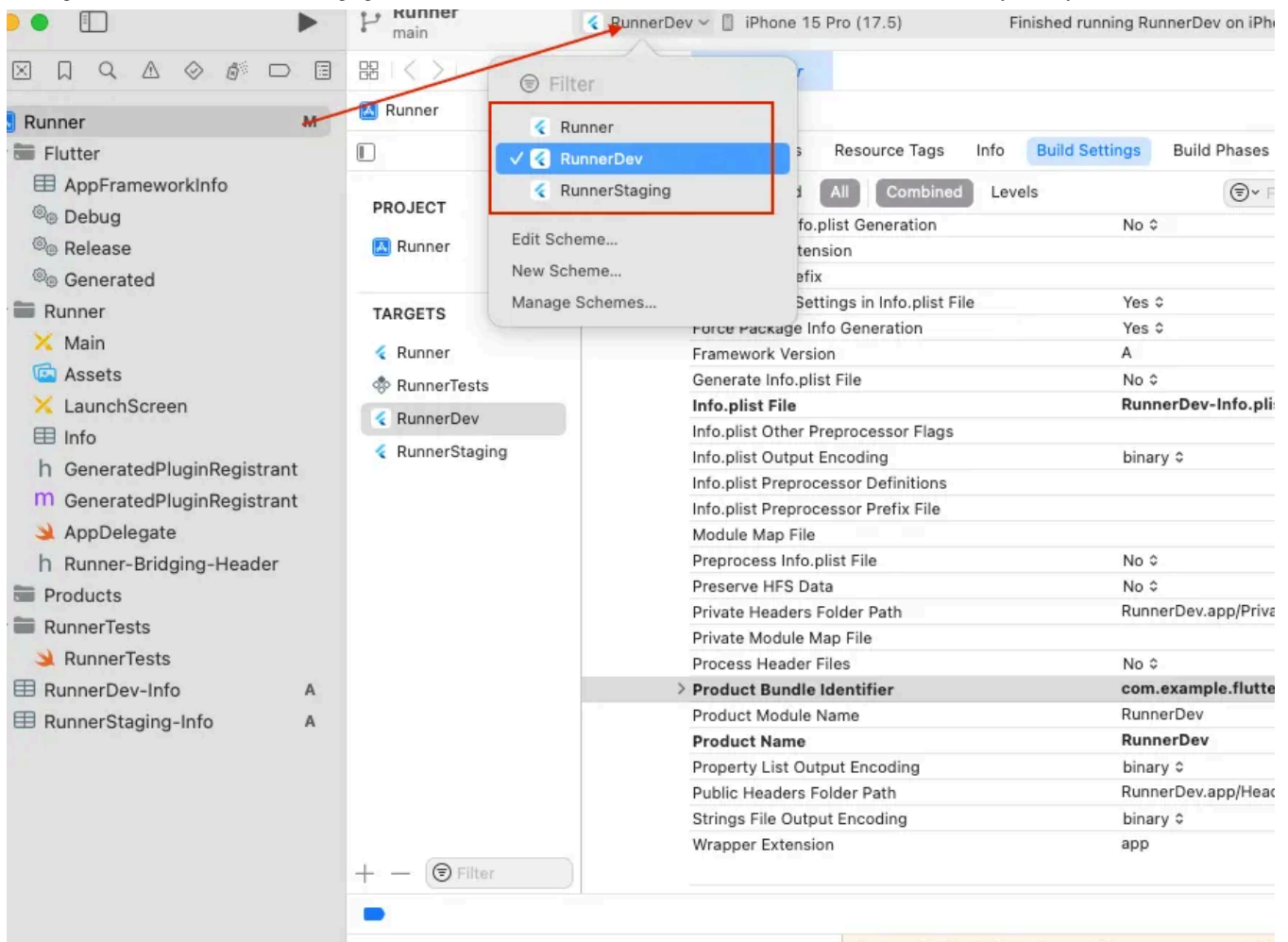


Time to Test!

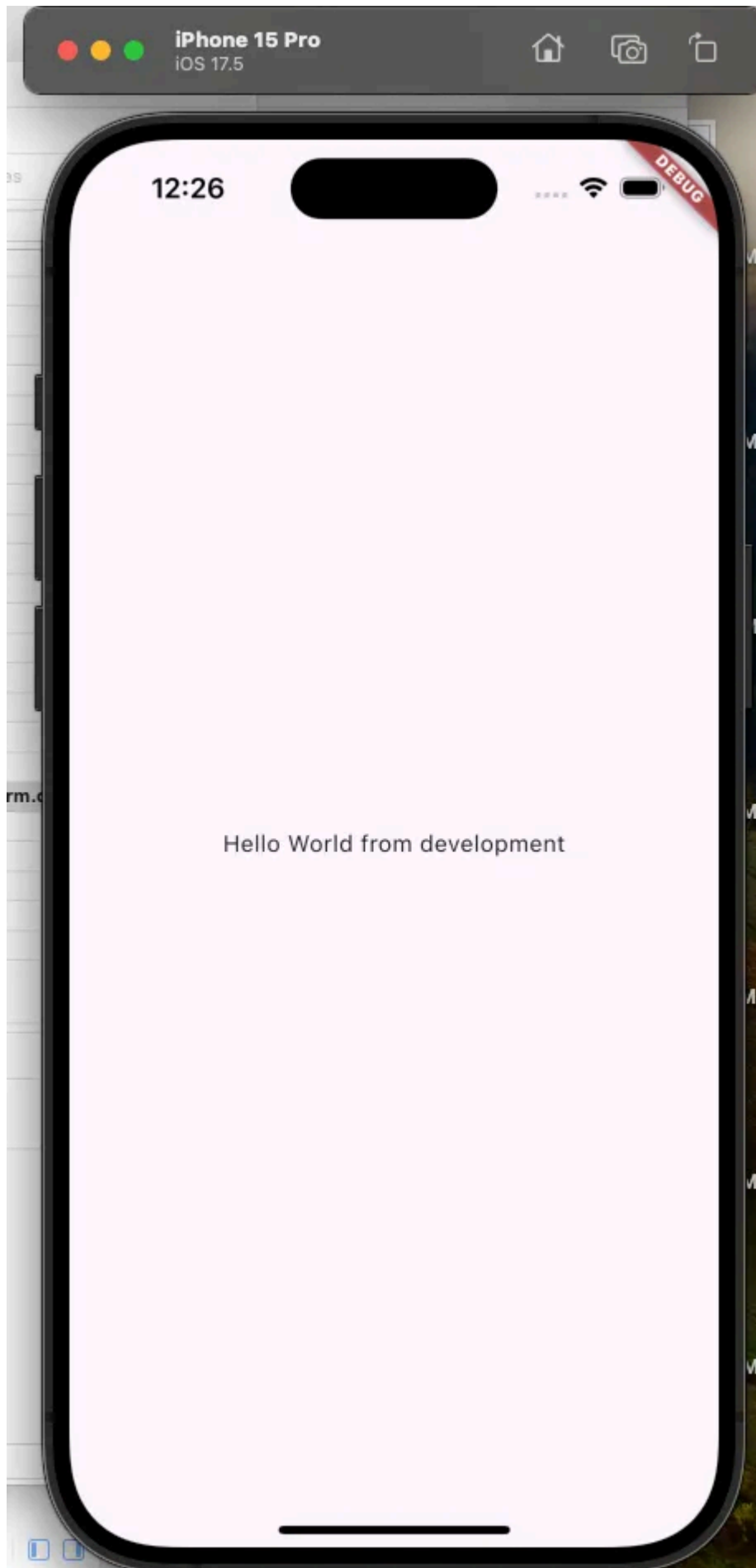
Update main.dart file:



Save all your changes and open your Flutter project in Xcode by navigating to the `ios` directory and opening the `Runner.xcworkspace` file. Click on the current Scheme shown in the screenshot below, select the Scheme you need to run, then click on the play (▶) button to run the app.



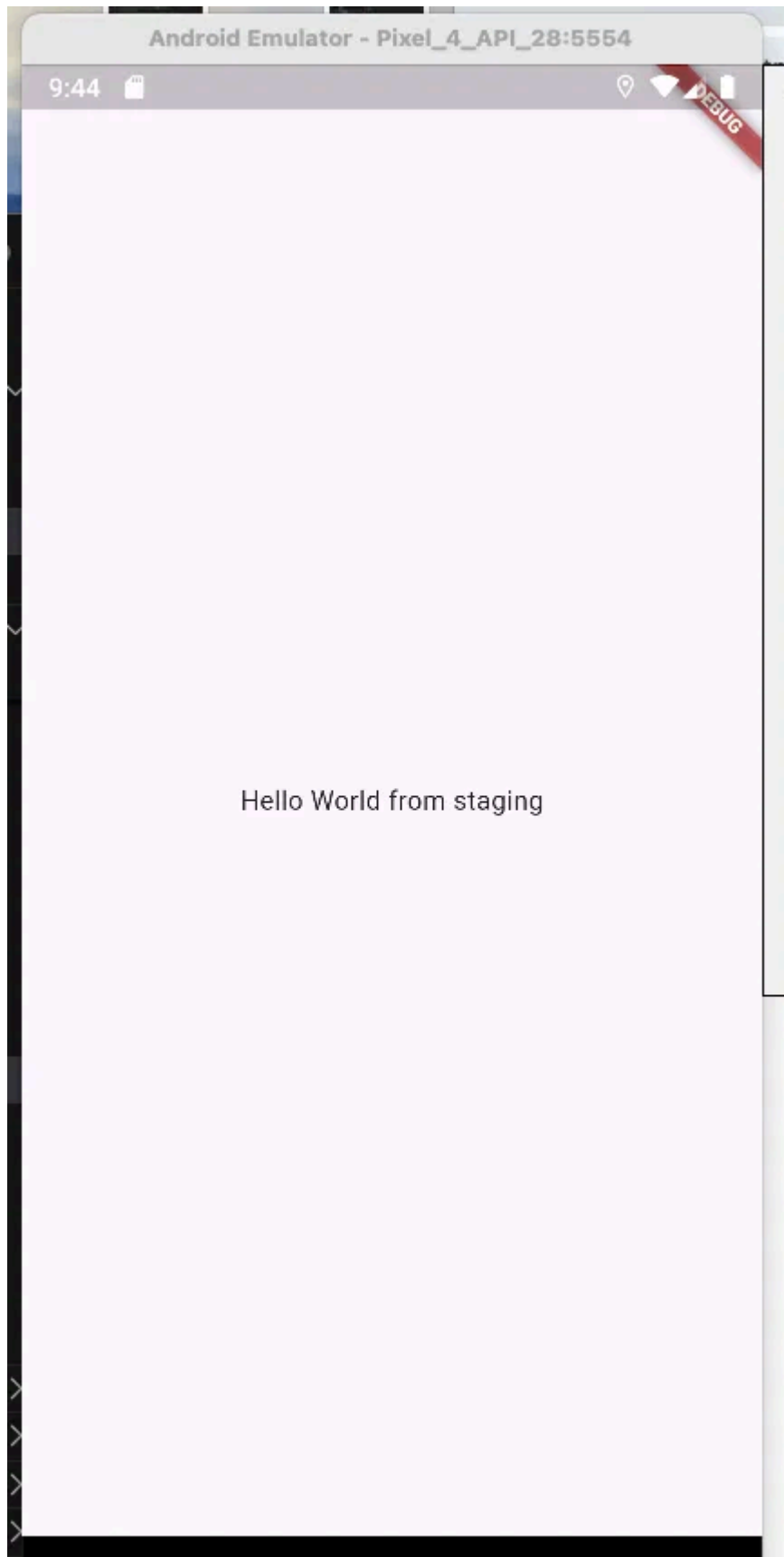
Yippee! Your app is now running. Now, you can repeat the same steps for staging and prod environments.



For Android:

Run this command in your terminal, and you can specify whatever flavor you need to run:

```
flutter run --flavor staging --dart-define=FLAVOR=staging
```



Other flavors can be run using:

```
flutter run --flavor dev --dart-define=FLAVOR=dev
```

```
flutter run --flavor prod --dart-define=FLAVOR=prod
```

Gbaski!

Congratulations! You've successfully created environment variables and flavors for an Android and iOS Flutter app.

P.S. If you'd like to see a video version of this article, feel free to drop a comment below! I'd love to hear your thoughts and create a visual walkthrough if there's enough interest.



Follow

Written by Sunday David

8 followers · 1 following

Consulting software engineer

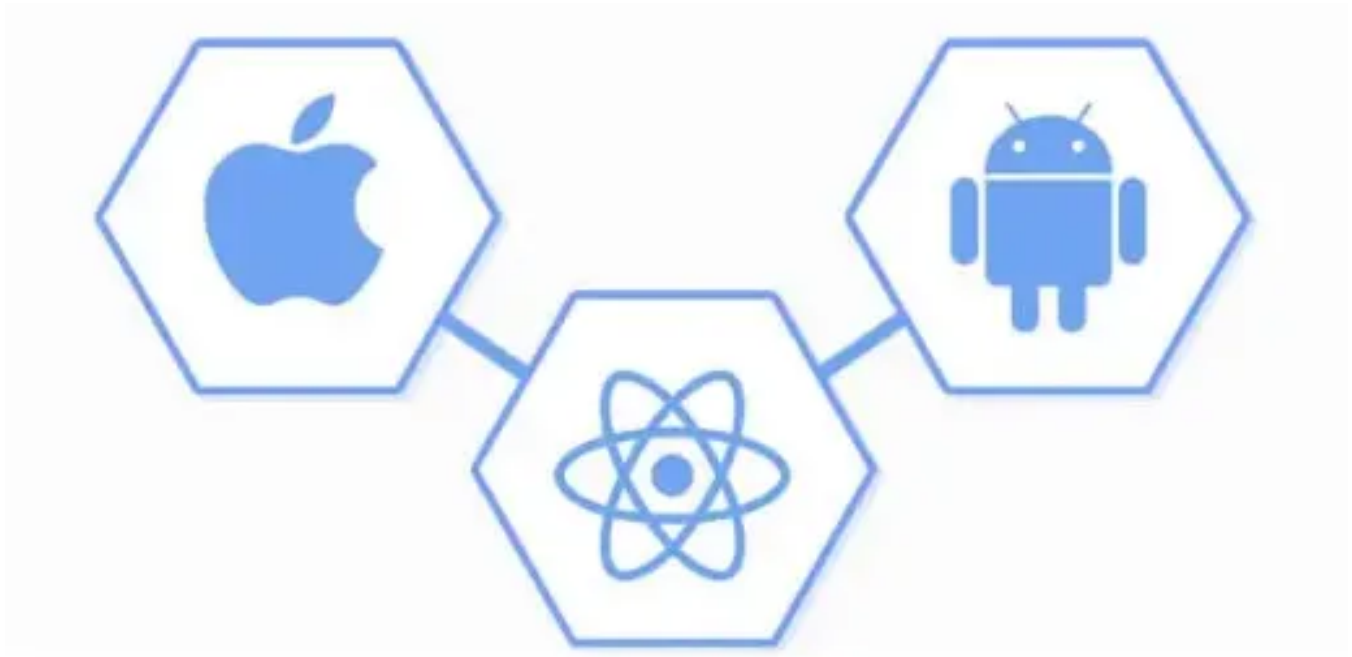
No responses yet



Write a response

What are your thoughts?

More from Sunday David



 Sunday David

Multiple Environments Setup in React Native for Android and iOS app

Setting up multiple environments in app development is crucial for comprehensive testing, seamless deployment, and overall robustness of...

Oct 7, 2023  340  6





Sunday David

Managing Multiple Firebase Projects in a React Native App

Managing multiple Firebase projects in a React Native app involves setting up and configuring Firebase for different environments, such as...

Oct 23, 2023



61



1



Sunday David

Why Social Media Needs a Content Rating System (And How to Build One)

A proposal for platform-native audience gating that protects users without killing creativity

Mar 1





Sunday David

Using Docker to Deploy Angular Universal (SSR) Application to Kubernetes

In this post, I'll explain the process of Using Docker to deploy Angular Universal (SSR) Application to Kubernetes. However, we'll be using...

Oct 1, 2022



See all from Sunday David

Recommended from Medium



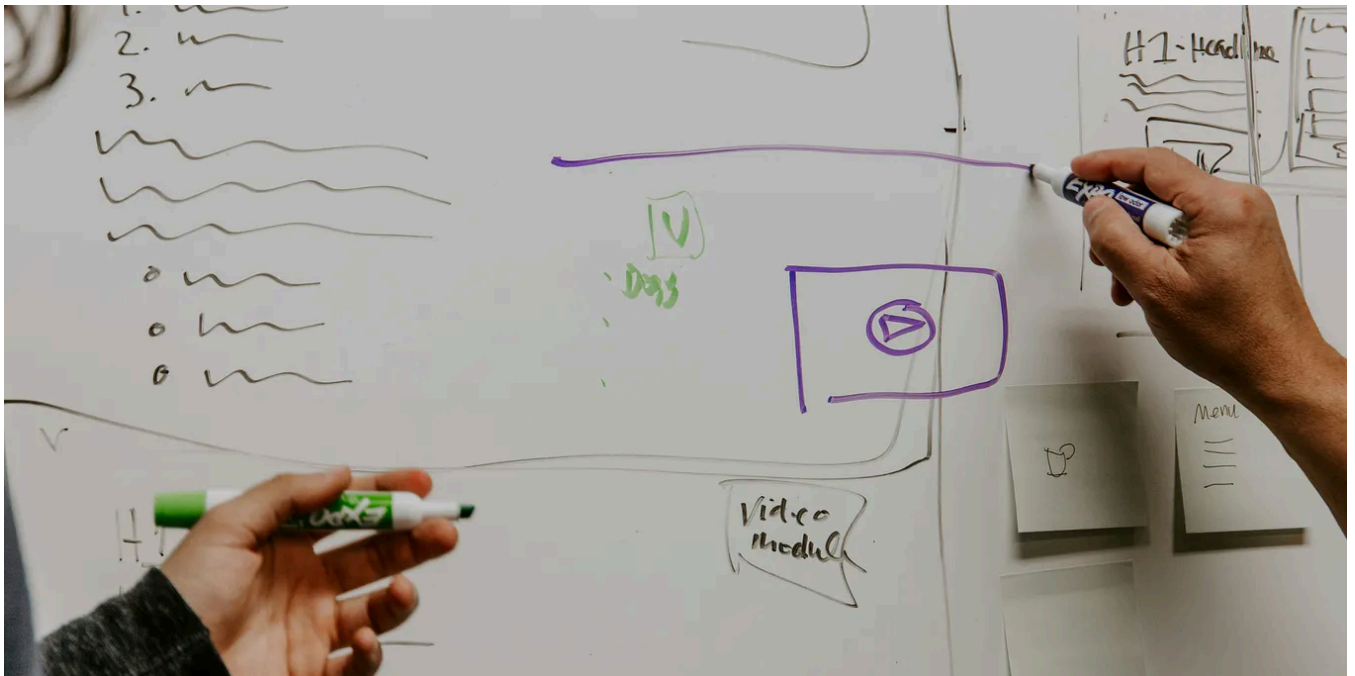
 In Easy Flutter by Pragnesh Palsana (APP DEVELOPER)

How I Built a Flutter App Using Claude Code in 2 Days (Full Workflow)

NON MEDIUM MEMBER LINK...

★ Mar 27 🖱️ 67 💬 9





 Shubham Singh

7 open-source iOS apps that will make you a better developer

Over the years, I've noticed something about how developers grow.

★ Apr 16 🖱 54 💬 1



7 Terminal Expert Skills



Jerry PM

7 Terminal Expert Skills

Every iOS Developer Should Master to Work Faster

★ 5d ago 🖱️ 17 💬 1



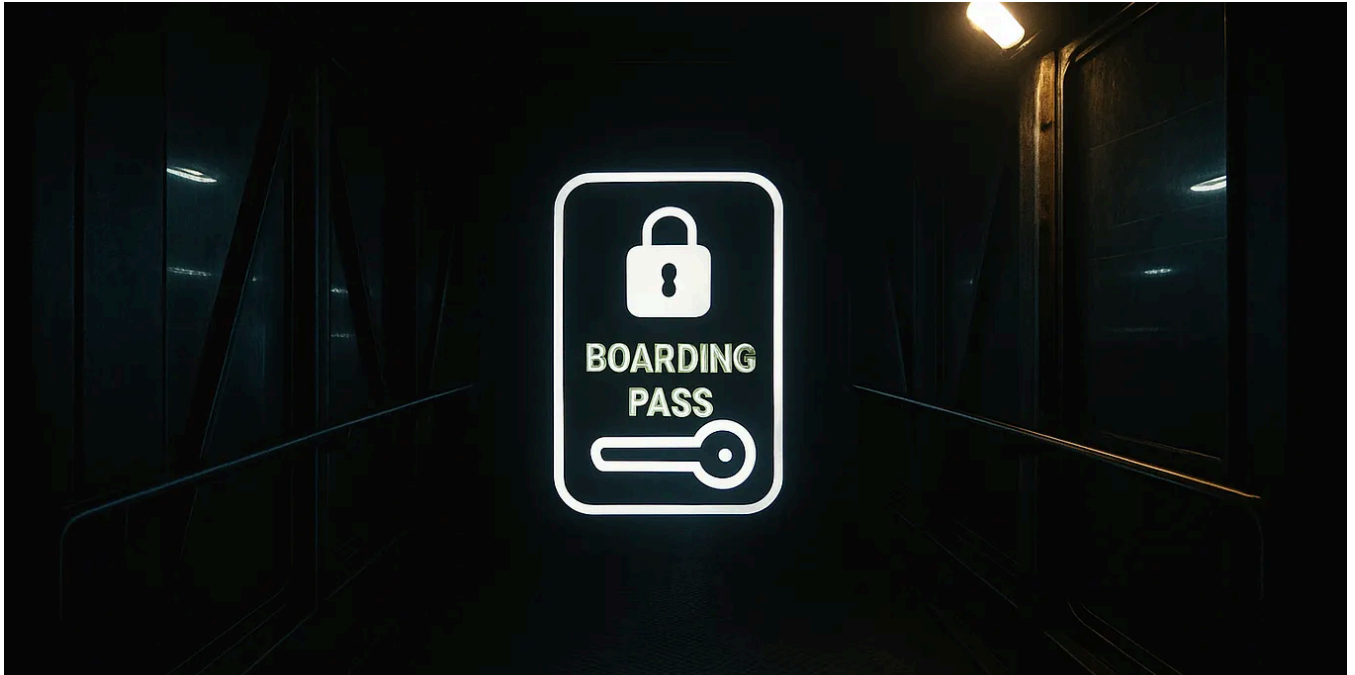
Type	Encrypted system database	Hardware-isolated coprocessor
Stores arbitrary data	Yes (passwords, tokens, etc.)	No — only cryptographic keys
Can store private keys?	Yes (but exportable)	Yes (non-exportable, most secure)
Biometric authentication	Supported via <code>AccessControl</code> flags	Native + strongest
Speed	Fast	Slower (due to hardware boundary)
Cross-device sync	Yes with iCloud Keychain	No (keys stay on device)
App groups support	Yes	Yes (if key is stored in Keychain but backed by SE)
Export key material	Possible	Impossible (forbidden by design)

AmitAswal

Keychain vs Secure Enclave—A Complete, Practical Guide for iOS Developers

Security on iOS is already top-tier, but when you're dealing with sensitive user data—tokens, passwords, cryptographic keys—you must...

Nov 18, 2025



Wesley Matlock



iOS App Attest + DeviceCheck: Building Real Trust Into Your App (Without Losing Your Mind)

App Attest + DeviceCheck: the backstage crew keeping your app from getting wrecked.

Nov 24, 2025



In Dev Genius by Simra Husain

Flutter Jank You Can't Fix With Isolates (And What Actually Works)

Stop optimizing your Dart code. The real bottleneck is in your rendering pipeline.

★ Dec 3, 2025 🖱 59 💬 3



See more recommendations